



RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de
Technologie



RAPPORT DE PROJET DE FIN D'ANNÉE

PFA — Année universitaire 2025–2026

Sujet :

DevSecOps avec Service Mesh : Sécurité Avancée Interservices

Réalisé par : Cherif Farah, Jouili Rim, Cherif Farah, Mizaoui Tasnim

Supervisé par : Mme. Ben Yahia Saloua

Examiné par : Mme. Mliki Hazar

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrante, **Mme. Ben Yahya Saloua** , pour sa disponibilité, ses précieux conseils et la rigueur scientifique qu'elle nous a transmise tout au long de ce projet.

Nos remerciements s'adressent également à l'ensemble du corps enseignant du département pour la formation de qualité dispensée durant notre cursus.

Nous sommes très reconnaissants envers les membres du jury honorable d'avoir pris le temps d'évaluer notre projet de fin d'année et pour leurs commentaires éclairants.

Enfin, nous remercions chaleureusement nos familles et amis pour leur soutien moral et leur encouragement constant.

Résumé

La multiplication des architectures microservices et le rythme croissant des cycles de livraison logicielle soulèvent de nouveaux défis en matière de sécurité. Le paradigme **DevSecOps** propose d'intégrer la sécurité à chaque étape du cycle de développement, tandis que le **Service Mesh** offre une couche d'infrastructure transparente pour sécuriser les communications interservices.

Ce rapport présente une étude approfondie de la combinaison de ces deux approches, en mettant en lumière leur complémentarité pour garantir une sécurité avancée dans les environnements distribués. Nous proposons notamment un prototype opérationnel déployé sur **Kubernetes** avec **Istio**, intégrant le chiffrement mutuel TLS (mTLS), l'authentification par jetons JWT et le contrôle d'accès par rôles (RBAC), validé par des tests DAST et une analyse de menaces selon la méthodologie STRIDE.

Mots-clés : DevSecOps, Service Mesh, Istio, Kubernetes, mTLS, DAST, SAST, microservices, sécurité, STRIDE, RBAC, JWT.

Table des matières

Remerciements	
Résumé	i
Acronymes et Termes Techniques	vi
Outils et Technologies	vii
Introduction Générale	1
1 DevSecOps et Pratiques de Sécurité	3
1.1 Objectifs du projet	3
1.2 Introduction au DevSecOps	4
1.2.1 Du DevOps au DevSecOps : genèse et évolution	4
1.2.2 Les principes fondateurs du DevSecOps	5
1.3 Les types de tests de sécurité (SAST, DAST, IAST, RASP)	5
1.3.1 SAST Static Application Security Testing	5
1.3.2 DAST Dynamic Application Security Testing	6
1.3.3 IAST Interactive Application Security Testing	7
1.3.4 RASP Runtime Application Self-Protection	7
1.3.5 Tableau comparatif des quatre approches	8
1.4 Focus DAST : enjeux dans les architectures microservices	8
1.4.1 Spécificités des architectures microservices	8
1.4.2 Stratégies DAST adaptées aux microservices	9
1.4.3 Limites actuelles et pistes d'amélioration	10
2 Service Mesh Présentation et Implémentation	12
2.1 Qu'est-ce qu'un Service Mesh ?	12
2.2 Architecture et Composants Fondamentaux	12
2.2.1 Le Pattern Sidecar	12
2.2.2 Composants Istio Déployés	13
2.3 Solutions Service Mesh — Comparaison	14
2.4 Apports du Service Mesh dans les Pratiques DevSecOps	14
2.5 Architecture du Prototype Déployé	15
2.5.1 Les Deux Microservices	15
2.5.2 Configuration Istio en Six Couches	16
2.5.3 Flux de Requête et Gestion des Certificats	17
3 Threat Modeling et Analyse de Sécurité	20

3.1	Méthodologie STRIDE	20
3.2	Architecture du Système Analysé	20
3.3	Analyse des Six Catégories STRIDE	21
3.3.1	Spoofing : Usurpation d'Identité	21
3.3.2	Tampering : Altération de Données en Transit	21
3.3.3	Repudiation : Non-Traçabilité des Actions	22
3.3.4	Information Disclosure : Divulcation d'Information	22
3.3.5	Denial of Service : Dénier de Service	22
3.3.6	Elevation of Privilege : Élévation de Privilèges	23
3.4	Synthèse des Risques et Contre-mesures	23
4	Prototype et Validation Expérimentale	26
4.1	Architecture du Prototype	26
4.1.1	Environnement de Déploiement	26
4.1.2	Services Déployés	26
4.2	Les Six Couches de Sécurité Implémentées	27
4.3	Analyse Comparative	28
4.4	Résultats Détaillés par Scénario	29
4.4.1	T1 — Accès à <code>/charge</code> sans JWT (Elevation of Privilege)	30
4.4.2	T2 — Pod sans Sidecar vers <code>payment-api</code> (Spoofing)	31
4.4.3	T3 et T4 — Cross-namespace et identité SPIFFE non autorisée	32
4.4.4	T5 et T6 — JWT Corrompu et Issuer Non Reconnu (Tampering)	33
4.4.5	T7 — Audit Trail et Non-Répudiation (Repudiation)	35
4.4.6	T8 — Vérification du Chiffrement mTLS (Information Disclosure)	37
4.4.7	T9 — Abus de Jeton ServiceAccount (Information Disclosure / Elevation of Privilege)	37
4.4.8	T10 et T11 — DoS et Rate Limiting	39
4.5	Observabilité avec Kiali et Grafana	40
4.5.1	Tableau des Vues d'Observabilité	40
4.5.2	Captures Kiali	41
4.5.3	Captures Grafana	44
4.6	Synthèse des Apports DevSecOps	45
	Bibliographie	47

Table des figures

2.1	Architecture du prototype Service Mesh — Kubernetes/Minikube avec Istio .	15
2.2	Diagramme de séquence : flux complet d'une requête /charge en trois phases — (1) validation JWT à l'Ingress Gateway, (2) routage mTLS vers payment-api avec retry automatique (max 3), (3) appel sécurisé à fraud-detection avec vérification de l'identité SPIFFE	18
2.3	Cycle de vie des certificats mTLS Istio : de l'injection du sidecar à l'utilisation du certificat, avec rotation automatique toutes les 24h.	19
4.1	Tableau de bord Kiali — visualisation du flux de trafic entre l'Ingress Gateway et le service	30
4.2	Journal d'accès Envoy — log de refus RBAC	31
4.3	Pod attacker déployé hors du mesh Istio — absence de sidecar Envoy . . .	31
4.4	Tentative d'accès depuis le pod attacker — connexion rejetée par mTLS (Connection reset by peer)	32
4.5	Kiali : flux fake-api → fraud-detection bloqué à 100 % d'erreurs.	33
4.6	Kiali : détail du trafic entrant sur fraud-detection — 100 % d'erreurs. . . .	33
4.7	Test T5 — envoi d'un JWT à signature corrompue, rejet HTTP 401 et trafic en erreur visible dans Kiali	34
4.8	Logs Envoy — refus jwt_authn_access_denied{Jwt_verification_fails} pour chaque requête avec signature invalide	34
4.9	Flux JWT : validation par RequestAuthentication et contrôle d'accès par AuthorizationPolicy au niveau de l'Istio Ingress Gateway.	35
4.10	Vue topologique du mesh — phase baseline (88,38 % de succès).	41
4.11	Workload attacker-cross-ns — sidecar manquant, trafic nul.	42
4.12	Arête HTTP gateway → payment-api — mTLS activé, principaux SPIFFE visibles.	43
4.13	Grafana Istio Service Dashboard — pic de latence et sources de trafic distinctes à 11 :40.	44
4.14	Grafana Istio Mesh Dashboard — 8 ops/s, 1,67 ops/s en 4xx, inventaire Istio confirmé.	45

Liste des tableaux

1.1	Comparaison des approches de tests de sécurité	8
2.1	Comparaison des principales solutions Service Mesh	14
3.1	Scénarios d’attaque STRIDE et contre-mesures Istio	24
4.1	Services déployés dans le prototype	27
4.2	Les six couches de sécurité du prototype	28
4.3	Comparaison avant/après application des politiques Istio	29
4.4	Vues d’observabilité Kiali et Grafana	40
4.5	Apports DevSecOps mesurés sur le prototype	46

Acronymes et Termes Techniques

API	Application Programming Interface (Interface de programmation applicative)
CI/CD	Continuous Integration / Continuous Deployment (Intégration Continue / Déploiement Continu)
DAST	Dynamic Application Security Testing (Test de sécurité dynamique des applications)
DNS	Domain Name System (Système de noms de domaine)
gRPC	Google Remote Procedure Call (Protocole d'appel de procédure à distance)
IAST	Interactive Application Security Testing (Test de sécurité interactif des applications)
IDS	Intrusion Detection System (Système de détection d'intrusion)
JWT	JSON Web Token (Jeton d'authentification web au format JSON)
mTLS	Mutual Transport Layer Security (Chiffrement TLS mutuel entre services)
OWASP	Open Worldwide Application Security Project
RASP	Runtime Application Self-Protection (Auto-protection applicative à l'exécution)
RBAC	Role-Based Access Control (Contrôle d'accès basé sur les rôles)
SAST	Static Application Security Testing (Test de sécurité statique des applications)
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege (Méthodologie de modélisation des menaces)
TLS	Transport Layer Security (Protocole de sécurisation des communications)
YAML	YAML Ain't Markup Language (Format de sérialisation de données)

Outils et Technologies

Envoy	Proxy sidecar haute performance utilisé comme data plane dans Istio, responsable de l'interception et du traitement de tout le trafic interservices.
Grafana	Outil de visualisation et de création de tableaux de bord, utilisé pour l'observabilité des métriques de l'infrastructure.
Istio	Service Mesh open source déployé sur Kubernetes, fournissant le chiffrement mTLS, la gestion du trafic, l'authentification JWT et le contrôle d'accès RBAC de manière transparente.
Jaeger	Système de traçage distribué permettant de suivre les requêtes à travers les différents microservices du cluster.
Kiali	Console d'observabilité dédiée à Istio, offrant une visualisation graphique de la topologie des services et de leurs flux de communication.
Kubernetes	Plateforme d'orchestration de conteneurs permettant l'automatisation du déploiement, de la mise à l'échelle et de la gestion des applications conteneurisées.
Minikube	Outil permettant d'exécuter un cluster Kubernetes mono-nœud en local, utilisé dans le cadre du prototype de validation.
OWASP ZAP	Zed Attack Proxy — outil de test de sécurité dynamique (DAST) open source, utilisé pour l'analyse des vulnérabilités des API exposées.
Prometheus	Système de collecte et de surveillance de métriques, intégré nativement à Istio pour l'observabilité du trafic interservices.

Introduction Générale

Contexte et Problématique

L'industrie du logiciel connaît depuis une décennie une transformation profonde, portée par deux évolutions majeures. D'une part, le **passage aux architectures microservices** a fragmenté les applications monolithiques en un ensemble de services indépendants, chacun pouvant être développé, déployé et mis à l'échelle individuellement. D'autre part, l'adoption généralisée des pratiques **DevOps** a raccourci drastiquement les cycles de livraison, permettant de passer de plusieurs déploiements par an à plusieurs déploiements par jour.

Si ces deux tendances ont considérablement amélioré l'agilité et la résilience des systèmes, elles ont simultanément introduit une **surface d'attaque étendue** et difficile à maîtriser. Un système composé de dizaines, voire de centaines de microservices communicants génère autant de canaux de communication potentiellement vulnérables, et la vitesse de déploiement laisse peu de place aux audits de sécurité traditionnels, souvent relégués en fin de cycle.

Constat fondamental : Selon le rapport *State of DevOps 2023*, plus de 60 % des incidents de sécurité en production dans les environnements cloud-natifs sont liés à des communications interservices non sécurisées ou à des configurations d'accès insuffisamment contrôlées.

Face à ce constat, la communauté technique a progressivement convergé vers deux approches complémentaires. Le paradigme **DevSecOps** préconise d'intégrer la sécurité dès les premières phases du développement ; le principe du *Shift Left Security* en automatisant les contrôles de sécurité au sein des pipelines CI/CD. Le **Service Mesh**, quant à lui, propose une couche d'infrastructure dédiée à la gestion des communications interservices, offrant nativement des mécanismes de chiffrement, d'authentification et d'observabilité sans modifier le code applicatif.

La problématique centrale de ce projet est donc la suivante :

Dans quelle mesure l'intégration d'un Service Mesh au sein d'une démarche DevSecOps permet-elle de renforcer significativement la sécurité des communications interservices dans les architectures microservices, et quelles sont les conditions d'une telle intégration ?

Ce rapport est organisé en quatre chapitres complémentaires, articulés de manière progressive depuis les fondements conceptuels jusqu'à la validation expérimentale.

Le Chapitre 1 pose les fondements du paradigme DevSecOps en retraçant son émergence depuis le mouvement DevOps et en détaillant ses principes fondateurs, notamment le *Shift Left Security*. Il dresse ensuite un panorama structuré des quatre grandes familles de tests de

sécurité :SAST, DAST, IAST et RASP en mettant en évidence leurs complémentarités et leurs limites respectives. Une attention particulière est portée au DAST dans les architectures microservices, dont les défis spécifiques ; explosion de la surface d'attaque, dynamique de l'infrastructure, inaccessibilité des flux internes constituent la problématique centrale qui motive l'ensemble du projet.

Le Chapitre 2 introduit le concept de Service Mesh comme réponse architecturale aux limites identifiées au chapitre précédent. Il présente l'architecture fondamentale du Service Mesh, structurée autour du Data Plane et du Control Plane, ainsi que les composants clés d'Istio : `istiod`, les proxies Envoy, `istio-ingressgateway`, Kiali et la stack d'observabilité Prometheus/Grafana. Un panorama comparatif des principales solutions du marché (Istio, Linkerd, Consul, Cilium) justifie le choix d'Istio pour ce projet, avant d'analyser les apports concrets du Service Mesh dans les pratiques DevSecOps.

Le Chapitre 3 applique la méthodologie STRIDE au prototype de système de paiement, afin de modéliser les menaces pesant sur une architecture microservices. Pour chacune des six catégories ; Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service et Elevation of Privilege ; les scénarios d'attaque sont analysés en comparant le comportement du système avec et sans Service Mesh. Le chapitre se conclut par une synthèse de douze scénarios d'attaque documentés, mettant en correspondance chaque menace avec le mécanisme Istio responsable de sa neutralisation.

Le Chapitre 4 présente la validation expérimentale de l'ensemble des analyses théoriques. Il décrit l'architecture du prototype déployé sur Minikube avec Istio, détaille l'implémentation des six couches de sécurité mTLS, JWT, RBAC, SPIFFE, NetworkPolicy et TLS Gateway et documente les résultats obtenus pour chacun des douze scénarios d'attaque. Les tableaux de bord Kiali et Grafana sont exploités pour démontrer l'observabilité en temps réel de la posture de sécurité, et une synthèse quantifiée mesure les apports concrets du Service Mesh par rapport à un déploiement Kubernetes sans politiques Istio.

Chapitre 1

DevSecOps et Pratiques de Sécurité

Introduction

L'introduction générale a mis en évidence deux constats fondamentaux : d'une part, les architectures microservices démultiplient la surface d'attaque et rendent les communications interservices difficiles à sécuriser [16] ; d'autre part, le Service Mesh s'impose comme une infrastructure complémentaire capable d'adresser les angles morts des approches de test classiques [1].

Ce chapitre construit le socle méthodologique qui structure l'ensemble du projet. Il présente successivement le paradigme DevSecOps (section 1.2), les quatre grandes familles de tests de sécurité applicative SAST, DAST, IAST et RASP en mettant en évidence leurs complémentarités et leurs limites respectives (section 1.3), puis les enjeux spécifiques du DAST dans les architectures microservices, dont les limites motivent directement l'approche hybride proposée (section 1.4).

1.1 Objectifs du projet

Sur la base de ces constats, ce projet vise à concevoir et valider une approche de sécurité hybride intégrant le Service Mesh dans une démarche DevSecOps, afin de couvrir l'ensemble des dimensions de sécurité d'une architecture microservices aussi bien les flux externes que les communications interservices internes [5].

Les objectifs poursuivis s'articulent autour de trois axes :

Cadre méthodologique DevSecOps. Maîtriser le paradigme DevSecOps et les quatre familles de tests de sécurité SAST, DAST, IAST, RASP afin d'identifier les outils les mieux adaptés à chaque phase du cycle de développement [14], et de caractériser précisément les limites du DAST conventionnel face à la dynamique des architectures microservices.

Architecture de sécurité proposée. Concevoir une architecture hybride combinant Istio Service Mesh et un pipeline DAST automatisé (OWASP ZAP) [3], couvrant à la fois les flux

north-south (trafic externe) et *east-west* (communications interservices internes), et intégrée dans une pipeline CI/CD reproductible déclenchée à chaque déploiement en environnement de staging.

Validation expérimentale. Déployer un prototype représentatif un système de paiement microservices sur un cluster Kubernetes réel (Minikube avec Istio) [11], évaluer l’apport concret du Service Mesh face à douze scénarios d’attaque couvrant les six catégories STRIDE [4], et mesurer les limites restantes de l’approche.

Architecture proposée. L’approche développée dans ce projet repose sur trois piliers complémentaires : **(1)** le Service Mesh (Istio) comme infrastructure de sécurité transparente assurant mTLS, contrôle d’accès et observabilité [1] ; **(2)** le DAST interne (OWASP ZAP) comme mécanisme d’évaluation continue de la posture de sécurité, étendu aux flux interservices [3] ; **(3)** la pipeline DevSecOps comme cadre d’intégration et d’automatisation de l’ensemble [14].

1.2 Introduction au DevSecOps

1.2.1 Du DevOps au DevSecOps : genèse et évolution

Le mouvement DevOps, apparu au tournant des années 2010, visait à supprimer le cloisonnement traditionnel entre les équipes de développement (*Development*) et d’exploitation (*Operations*) [15]. En automatisant les pipelines d’intégration et de déploiement continus (CI/CD), il a permis d’accélérer considérablement la livraison logicielle tout en améliorant la stabilité des systèmes [14,13].

Cependant, cette accélération a mis en évidence une faille structurelle : la sécurité restait souvent une préoccupation tardive, traitée en phase finale par des équipes spécialisées travaillant en silo. Ce modèle, désigné par l’expression « Security as a Gatekeeper », se révèle incompatible avec des rythmes de déploiement quotidiens, car il crée des goulots d’étranglement et laisse des vulnérabilités non détectées atteindre la production [14].

Le concept de DevSecOps- contraction de *Development*, *Security* et *Operations* émerge comme réponse à cette inadéquation. Il s’agit d’une culture, d’un ensemble de pratiques et d’outils visant à intégrer la sécurité à chaque étape du cycle de vie logiciel, en partageant la responsabilité de la sécurité entre toutes les équipes [15,14].

Le DevSecOps est une approche organisationnelle et technique qui intègre la sécurité comme une composante intrinsèque et continue du cycle de développement et d’exploitation logicielle, par opposition à un contrôle ponctuel en fin de cycle.

1.2.2 Les principes fondateurs du DevSecOps

Le DevSecOps repose sur quatre piliers interdépendants :

1. **Automatisation de la sécurité.** Les contrôles de sécurité (analyse statique du code, scan de dépendances, tests dynamiques) sont intégrés directement dans les pipelines CI/CD et s'exécutent à chaque commit, sans intervention manuelle systématique [6].
2. **Collaboration et responsabilité partagée.** Les développeurs, les ingénieurs DevOps et les équipes de sécurité (SecOps) travaillent conjointement dès la phase de conception. La sécurité n'est plus la propriété exclusive d'une équipe spécialisée [14].
3. **Observabilité et retour d'information rapide.** Des mécanismes de monitoring, de logging et d'alerting permettent de détecter et de corriger rapidement les incidents de sécurité en production [13].
4. **Infrastructure as Code (IaC) sécurisée.** La configuration des infrastructures est déclarée sous forme de code versionné, soumis aux mêmes revues et analyses de sécurité que le code applicatif [6].

1.3 Les types de tests de sécurité (SAST, DAST, IAST, RASP)

La sécurité applicative mobilise plusieurs familles de techniques de test, chacune offrant un angle d'observation distinct sur la posture de sécurité d'un système [2]. Nous présentons ici les quatre approches majeures.

1.3.1 SAST Static Application Security Testing

Le SAST (test de sécurité statique) analyse le code source, le bytecode ou le code binaire d'une application **sans l'exécuter**, afin d'identifier des vulnérabilités telles que les injections SQL, les dépassements de tampon, ou les mauvaises pratiques cryptographiques [6].

Principes de fonctionnement. Le SAST opère par analyse syntaxique et sémantique du code. Il construit généralement un *Abstract Syntax Tree* (AST) et effectue une analyse du flux de données (*taint analysis*) pour tracer le cheminement des données non fiables depuis leurs sources jusqu'aux points d'utilisation potentiellement dangereux (*sinks*) [6].

Avantages.

— Détection précoce des vulnérabilités, dès la phase de codage.

- Couverture exhaustive du code, y compris les chemins d'exécution rarement empruntés.
- Intégration aisée dans les IDE et les pipelines CI [14].

Limites.

- Taux élevé de faux positifs, nécessitant une validation humaine.
- Incapacité à détecter les vulnérabilités de configuration ou les failles logiques dépendant du contexte d'exécution.
- Difficulté à analyser les dépendances tierces compilées.

Outils représentatifs. SonarQube, Semgrep, Checkmarx, Fortify, Bandit (Python), SpotBugs (Java).

1.3.2 DAST Dynamic Application Security Testing

Le DAST (test de sécurité dynamique) évalue la sécurité d'une application **en cours d'exécution**, en simulant des attaques réelles depuis l'extérieur, sans accès au code source [2]. Il adopte la perspective d'un attaquant (*black-box testing*).

Principes de fonctionnement. Le DAST interagit avec l'application via ses interfaces exposées (API REST, interfaces web, gRPC, etc.), en injectant des charges malveillantes (*payloads*) et en analysant les réponses pour détecter des comportements anormaux révélateurs de vulnérabilités : injections, authentification défaillante, exposition de données sensibles, etc. [2,3].

Avantages.

- Indépendant du langage et du framework applicatif.
- Détecte des vulnérabilités réelles dans l'environnement d'exécution, incluant celles issues de la configuration et des interactions entre composants [3].
- Faible taux de faux positifs par rapport au SAST.

Limites.

- Couverture partielle : ne peut tester que les fonctionnalités accessibles et les surfaces d'attaque exposées.
- Détection tardive dans le cycle de développement.
- Complexité accrue dans les architectures microservices, où la multitude des points d'entrée démultiplie la surface à couvrir [16].

Outils représentatifs. OWASP ZAP [3], Burp Suite, Nikto, Nuclei, Nessus.

1.3.3 IAST Interactive Application Security Testing

L'IAST (test de sécurité interactif) **combine les approches statique et dynamique** en instrumentant l'application à l'aide d'agents déployés dans son environnement d'exécution. Il analyse le comportement interne du code pendant les tests fonctionnels.

Principes de fonctionnement. Des agents IAST sont intégrés directement dans le runtime de l'application (JVM, CLR, interpréteur Python, etc.). Ils interceptent les flux d'exécution, tracent les données et détectent les vulnérabilités en observant les interactions réelles entre les composants, sans rejeter les requêtes légitimes.

Avantages.

- Très faible taux de faux positifs grâce à la corrélation contexte d'exécution / code source.
- Fonctionnement en temps réel, sans phase de scan dédiée.
- Adapté aux environnements de test d'intégration continue [14].

Limites.

- Surcharge de performance liée à l'instrumentation.
- Support limité à certains langages et frameworks.
- Couverture dépendante de l'exhaustivité des tests fonctionnels exécutés.

Outils représentatifs. Contrast Security, Seeker (Synopsys), Hdiv Security.

1.3.4 RASP Runtime Application Self-Protection

Le RASP (auto-protection applicative à l'exécution) est une technologie de sécurité **intégrée directement dans une application** ou son environnement d'exécution, capable de détecter et de bloquer des attaques en temps réel, sans intervention externe.

Principes de fonctionnement. Contrairement au DAST qui teste de l'extérieur, le RASP s'exécute à l'intérieur de l'application. Il intercepte les appels système, les requêtes de base de données et autres opérations sensibles, et peut les bloquer instantanément si un comportement malveillant est détecté, tout en émettant des alertes vers le SIEM.

Avantages.

- Protection en production, y compris contre des attaques zero-day [2].
- Réduction de la dépendance aux mises à jour du WAF.
- Contexte d'exécution précis, minimisant les faux positifs.

Limites.

- Impact sur les performances applicatives.
- Risque de faux positifs entraînant des interruptions de service.
- Complexité de déploiement et de maintenance.

Outils représentatifs. Sscreen (Datadog), Imperva RASP, OpenRASP.

1.3.5 Tableau comparatif des quatre approches

Table 1.1 – Comparaison des approches de tests de sécurité

Critère	SAST	DAST	IAST	RASP
Phase	Codage / build	Test / staging	Test / intég.	Production
Accès code	Requis	Non requis	Optionnel	Non requis
Perspective	White-box	Black-box	Gray-box	Interne
Faux positifs	Élevés	Faibles	Très faibles	Faibles
Couverture	Code complet	Surface exposée	Code exercé	Runtime
Impact perf.	Aucun	Faible	Moyen	Moyen-élevé
Microservices	Partiel	Complexe	Adapté	Adapté

Ce panorama des quatre approches met en évidence la complémentarité de leurs angles d’observation respectifs. Dans le cadre de ce projet, dont la problématique centrale porte sur la sécurisation des communications interservices dans les architectures microservices [16], c’est le **DAST** qui présente les enjeux les plus critiques et les limites les plus structurantes. En tant qu’approche *black-box* simulant des attaques réelles depuis l’extérieur, il est en première ligne face à la complexité des environnements microservices — mais c’est précisément là que ses lacunes sont les plus prononcées. La section suivante en dresse une analyse approfondie.

1.4 Focus DAST : enjeux dans les architectures microservices

1.4.1 Spécificités des architectures microservices

Une architecture microservices est un style architectural dans lequel une application est décomposée en un ensemble de services petits, autonomes et faiblement couplés, chacun responsable d’une capacité métier précise [16]. Ces services communiquent entre eux via des API bien définies, généralement sur des protocoles HTTP/REST ou gRPC, et peuvent être déployés indépendamment.

Cette architecture présente plusieurs caractéristiques qui complexifient significativement l'application du DAST :

Explosion de la surface d'attaque. Dans une application monolithique, la surface d'attaque exposée est concentrée sur un nombre limité de points d'entrée. Dans une architecture microservices, chaque service expose potentiellement sa propre API, multipliant d'autant les surfaces à tester [2,16]. Un système composé de 50 microservices peut exposer plusieurs centaines d'endpoints distincts, avec des contrats d'API hétérogènes.

Dynamacité de l'infrastructure. Les microservices s'exécutent dans des environnements hautement dynamiques (Kubernetes, par exemple) [11], où les instances de services apparaissent et disparaissent selon la charge. Les adresses IP et les ports sont attribués dynamiquement, rendant obsolète toute cible DAST définie statiquement. Un scanner DAST doit s'adapter en temps réel à cette topologie mouvante.

Communications interservices non exposées. Une partie significative des communications dans une architecture microservices est interne : les échanges entre services du backend ne sont pas directement accessibles depuis l'extérieur du cluster [11,16]. Les outils DAST conventionnels, conçus pour tester des interfaces web publiques, ne peuvent pas atteindre ces flux de communication internes sans mécanismes spécifiques.

Hétérogénéité des protocoles et des données. Les microservices peuvent utiliser des protocoles variés : REST/HTTP, gRPC, GraphQL, WebSocket, messaging asynchrone (Kafka, RabbitMQ) [16]. Les outils DAST doivent être capables de générer des payloads adaptés à chaque protocole et d'interpréter correctement les réponses correspondantes.

1.4.2 Stratégies DAST adaptées aux microservices

Face à ces défis, plusieurs stratégies permettent d'adapter le DAST aux architectures microservices :

Intégration dans la pipeline CI/CD. Plutôt que d'effectuer des scans ponctuels, le DAST est intégré comme une étape automatique de la pipeline, déclenchée à chaque déploiement en environnement de staging [14]. Des outils comme OWASP ZAP disposent de modes « headless » permettant cette intégration [3].

DAST orienté API. Des outils spécialisés dans le test de sécurité des API (comme 42Crunch, Traceable AI, ou les extensions REST/gRPC de ZAP [3]) permettent d'importer les spécifications OpenAPI/Swagger et de générer automatiquement des cas de test de sécurité couvrant l'ensemble des endpoints documentés.

Service Mesh comme point d’observation. L’adoption d’un Service Mesh ouvre une perspective nouvelle pour le DAST dans les microservices [1]. En opérant au niveau du sidecar proxy, le Service Mesh fournit une visibilité totale sur les communications interservices, y compris les flux internes au cluster, sans modification du code applicatif [1]. Cette caractéristique permet d’envisager un DAST interne, capable de tester la sécurité des échanges entre services — une dimension absente des approches DAST traditionnelles.

Synergie clé : Le Service Mesh ne se substitue pas au DAST, mais il en élargit considérablement le périmètre d’action dans les architectures microservices [1,16]. En fournissant un point d’interception centralisé et une observabilité totale du trafic interservices, il crée les conditions d’un DAST interne couvrant les vulnérabilités invisibles depuis l’extérieur du cluster.

1.4.3 Limites actuelles et pistes d’amélioration

Malgré les évolutions récentes, plusieurs limites persistent dans l’application du DAST aux architectures microservices :

- **Couverture incomplète des flux asynchrones :** les communications via des bus de messages (Kafka, RabbitMQ) restent difficiles à intercepter et à tester dynamiquement [16].
- **Manque de standardisation :** l’absence d’un format universel de description des API interservices limite la capacité des outils DAST à générer automatiquement des cas de test pertinents [2].
- **Complexité de l’environnement de test :** reproduire fidèlement un environnement microservices complet à des fins de test reste coûteux en ressources et en configuration [11].

Ces limites motivent l’exploration d’une approche hybride intégrant le Service Mesh comme infrastructure de sécurité complémentaire [5], objet des chapitres suivants.

Conclusion

Ce chapitre a posé les fondements conceptuels et méthodologiques sur lesquels repose ce projet. Nous avons d’abord formulé les objectifs du projet en trois axes : cadre méthodologique DevSecOps, architecture hybride proposée et validation expérimentale. Nous avons ensuite introduit le paradigme DevSecOps, en retraçant son émergence depuis le mouvement DevOps [15,14] et en détaillant ses principes fondateurs. Nous avons dressé un panorama structuré des quatre grandes familles de tests de sécurité SAST, DAST, IAST et RASP en mettant en évidence leurs complémentarités et leurs limites respectives [2,6].

La section finale a approfondi les enjeux spécifiques du DAST dans les architectures microservices, en identifiant les défis majeurs que soulève cette combinaison : explosion de la surface d'attaque, dynamicité de l'infrastructure [11], inaccessibilité des flux internes et hétérogénéité protocolaire [16].

Chapitre 2

Service Mesh Présentation et Implémentation

Introduction

Ce chapitre présente le concept de Service Mesh et son architecture fondamentale, avant d'analyser les principales solutions disponibles et de justifier le choix d'Istio pour ce projet [1]. L'architecture du prototype déployé est ensuite décrite comme réponse concrète aux limites du DAST identifiées au chapitre précédent.

2.1 Qu'est-ce qu'un Service Mesh ?

Un **Service Mesh** est une couche d'infrastructure dédiée à la gestion des communications service-à-service dans une architecture microservices [11]. Il fonctionne en déployant un proxy léger (*sidecar*) aux côtés de chaque instance de service. Ce proxy intercepte de manière transparente tout le trafic entrant et sortant, sans aucune modification du code applicatif [8].

Un Service Mesh est structuré en deux plans distincts : le **Data Plane**, constitué des proxies sidecars (typiquement Envoy) [8], qui traite le trafic en temps réel ; et le **Control Plane**, qui configure et orchestre les proxies via une API centralisée. Dans Istio, le Control Plane est incarné par le composant `istiod`, qui gère la distribution des certificats, la configuration des politiques et le service discovery [1].

2.2 Architecture et Composants Fondamentaux

2.2.1 Le Pattern Sidecar

Le pattern sidecar est le mécanisme clé du Service Mesh [11]. Pour chaque pod Kubernetes, un conteneur Envoy est automatiquement injecté lors du déploiement, dès que le namespace est annoté avec l'étiquette `istio-injection: enabled`. Des règles `iptables` redirigent tout le trafic réseau du pod vers ce proxy, qui devient ainsi le point de contrôle unique pour toutes

les communications [8].

Ce mécanisme est fondamentalement transparent pour l'application : le service métier continue d'envoyer et de recevoir des requêtes HTTP normales, tandis qu'Envoy gère en arrière-plan le chiffrement mTLS [9], la validation JWT, le contrôle d'accès et la collecte de métriques [1].

2.2.2 Composants Istio Déployés

- **istiod (Pilot + Citadel + Galley)** : Control plane unifié. Gère la distribution des certificats SPIFFE [7], la propagation des politiques et la configuration des proxies Envoy [1].
- **Envoy sidecar** : Proxy haute performance injecté dans chaque pod [11]. Implémente mTLS [9], routing, load balancing, circuit breaking, rate limiting et collecte des métriques [8].
- **istio-ingressgateway** : Point d'entrée externe du mesh. Gère la terminaison TLS pour le trafic Nord-Sud [1].
- **Kiali** : Console d'observabilité. Visualise la topologie du mesh, le statut mTLS, les métriques de trafic et les politiques actives [10].
- **Prometheus + Grafana** : Collecte et visualisation des métriques Envoy (latence, RPS, success rate, overhead TLS) [13].

2.3 Solutions Service Mesh — Comparaison

Table 2.1 – Comparaison des principales solutions Service Mesh

Critère	Istio	Linkerd	Consul	Cilium
mTLS automatique	✓ STRICT	✓	✓	✓
JWT / OIDC	✓ Natif	×	Limité	✓ via plugin
AuthorizationPolicy	✓ Granulaire	△ Basique	✓	△
SPIFFE	✓ Complet [7]	✓	✓	△ Partiel
Observabilité	✓ Kiali+Grafana [10]	✓ Dashboard	△	△
Complexité	Élevée	Faible	Moyenne	Moyenne
Choix projet	✓ Retenu	—	—	—

Istio a été retenu pour ce projet en raison de la richesse de son modèle de politiques de sécurité [1], notamment la gestion native du JWT, les **AuthorizationPolicies** basées sur les claims et la validation SPIFFE complète [7]. Ces fonctionnalités correspondent précisément aux besoins d'un système de paiement sécurisé nécessitant un contrôle d'accès granulaire [5].

2.4 Apports du Service Mesh dans les Pratiques Dev-SecOps

Le Service Mesh répond directement aux limitations du DAST identifiées au Chapitre 1, en ajoutant une dimension de sécurité runtime automatisée [1,14] :

- **Visibilité totale sur le trafic East-West** : chaque requête inter-service est interceptée, journalisée et métriquée par le proxy Envoy [8], rendant ce trafic inspectable et auditable.
- **Sécurisation automatique sans modification applicative** : mTLS [9], authentification et autorisation sont appliqués par l'infrastructure, indépendamment du code des services [1].
- **Politiques as Code** : les politiques de sécurité sont définies en YAML versionnable, intégrables dans le pipeline CI/CD et auditables [14].

- **Feedback continu** : Kiali et Grafana fournissent des métriques de sécurité en temps réel [10,13], permettant la détection d'anomalies comportementales.

Complémentarité avec le DAST : Le Service Mesh ne se substitue pas aux outils de test de sécurité existants [3], mais en élargit considérablement le périmètre d'action. En fournissant un point d'interception centralisé sur l'ensemble du trafic interservices [1], il crée les conditions d'une sécurité runtime couvrant les vulnérabilités invisibles depuis l'extérieur du cluster [5].

2.5 Architecture du Prototype Déployé

Pour valider concrètement les apports du Service Mesh, un prototype de système de paiement a été conçu et déployé sur Kubernetes (Minikube v1.28.0) avec Istio v1.21.0 [1,11]. Ce prototype constitue le terrain d'expérimentation de l'ensemble des analyses de sécurité développées aux chapitres suivants.

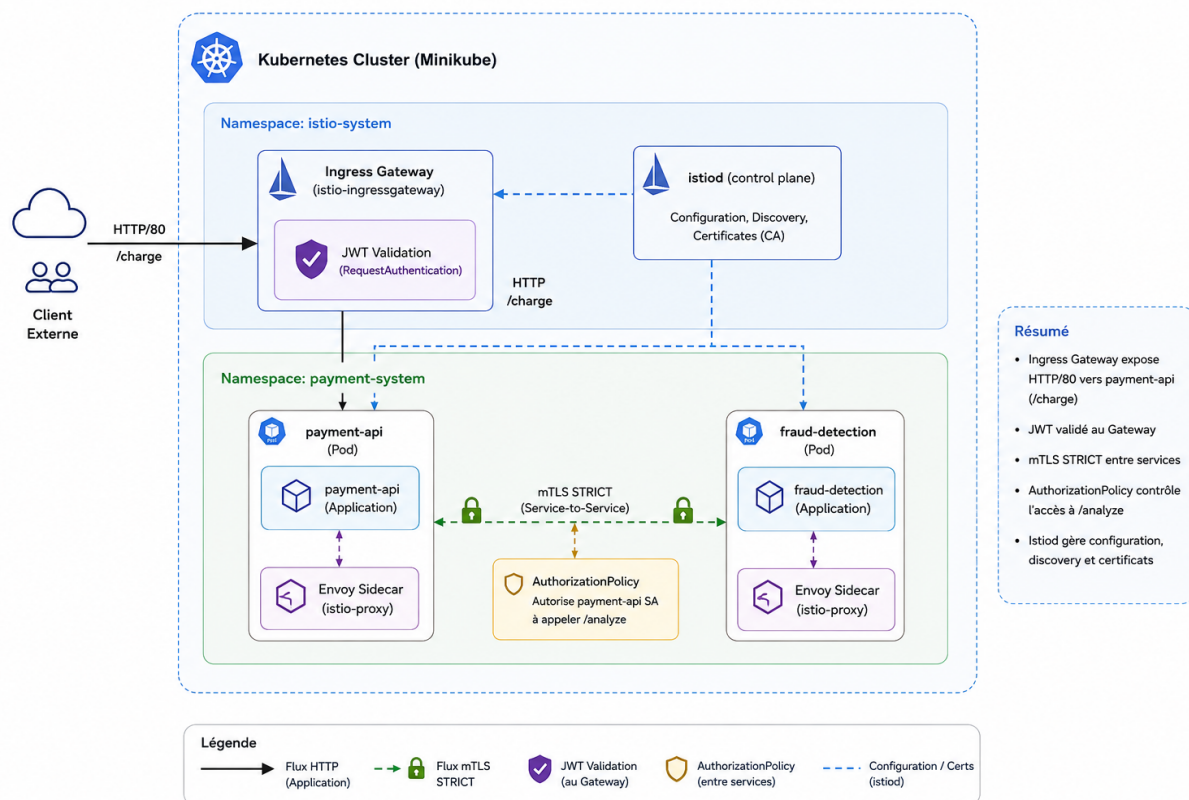


Figure 2.1 – Architecture du prototype Service Mesh — Kubernetes/Minikube avec Istio

2.5.1 Les Deux Microservices

Le système comprend deux microservices métier aux rôles complémentaires [16] :

- **Payment-API (Go, port 8080)** : service principal exposé via l’Ingress Gateway [1]. Il expose les endpoints `/health`, `/ready` et `/charge`, traite les requêtes de paiement et délègue l’analyse de risque à `fraud-detection` via un appel mTLS [9]. Il s’exécute sous le ServiceAccount `payment-api` avec 100m CPU et 128Mi RAM.
- **Fraud-Detection (Python/Flask, port 8080)** : service interne d’analyse de risque. Il expose les endpoints `/health`, `/ready` et `/analyze`, et n’est accessible que depuis `payment-api` via validation SPIFFE [7]. Il s’exécute sous le ServiceAccount `fraud-detection` avec les mêmes ressources allouées.

2.5.2 Configuration Istio en Six Couches

The security and traffic architecture relies on eight configuration resources deployed in the `mesh-config/` directory, split between Istio CRDs managed by the control plane (`istiod`) and a native Kubernetes resource operating at the CNI level [11].

Istio CRDs (managed by istiod)

- **VirtualServices (3)**
Routing rules with automatic retry (3×, `perTryTimeout=10s`) and a 30-second global timeout per inter-service call [1]. Two VirtualServices cover internal traffic (`payment-api`, `fraud-detection`); a third handles inbound traffic from the Gateway.
- **DestinationRules (2)**
Traffic policy for `payment-api` and `fraud-detection` : round-robin load balancing, connection pool capped at 100 simultaneous connections (`maxRequestsPerConnection=2`), and outlier detection that ejects a host after 5 consecutive 5xx errors (30-second interval, 30-second base ejection time) [1,8].
- **Gateway (1)**
External endpoint exposed on port 80 over HTTP [1]. **Limitation : no TLS termination is configured** — traffic between the client and the gateway is unencrypted. Production deployment requires a certificate and migration to HTTPS/443 [9].
- **PeerAuthentication (1)**
Enforces mTLS STRICT across all pods in the `payment-system` namespace [9,1]. Certificates are issued and rotated automatically by `istiod`. Any client without a valid sidecar or certificate is rejected at the Envoy proxy level.
- **RequestAuthentication (1)**
Validates JWT tokens (RS256) on `payment-api` [1]. Expected issuer : `https://testing.com`, audience : `payment-api`, with `forwardOriginalToken=true` so downstream services receive the original token. The verification key is provided via an inline JWKS — suitable

for a demo environment ; a remote JWKS endpoint should be used in production.

— **AuthorizationPolicies (2)**

Fine-grained zero-trust access control [5,17] :

- **payment-api** : `/health` and `/ready` are publicly accessible (liveness/readiness probes) ; `POST /charge` is allowed only from the `istio-system` namespace with JWT claim `role=customer`, or from service account `cluster.local/ns/payment-system/sa/fraud-detection`
- **fraud-detection** : `POST /analyze`, `/health`, and `/ready` are allowed only from service account `cluster.local/ns/payment-system/sa/payment-api`. All other access is denied by default [5].

— **EnvoyFilter (1)**

Local rate limiting applied directly on `istio-ingressgateway`, before any routing decision [8]. Token bucket : 20 tokens per second, refilled every second (`fill_interval=1s`), enforced on 100% of requests. This is a per-replica local limit — it does not constitute distributed global rate limiting.

Ressource Kubernetes native

- **NetworkPolicies (2)** [12] : mécanisme d'isolation réseau au niveau du CNI, indépendant du Service Mesh Istio. Cette couche reste effective même en cas d'absence ou de mauvaise configuration des sidecars, assurant une stratégie de défense en profondeur [5].

Les règles mises en place imposent :

- **Ingress** : seules les communications provenant du namespace `payment-system` (identifié par le label `name=payment-system`) sont autorisées sur le port TCP 8080.
- **Egress** : les sorties réseau sont limitées au namespace `payment-system` ainsi qu'au service DNS du cluster (`kube-dns`, port UDP 53).

2.5.3 Flux de Requête et Gestion des Certificats

Le flux complet d'une requête de paiement traverse les étapes suivantes : le client envoie une requête HTTP `/charge` à l'Ingress Gateway (port 31629), qui vérifie l'autorisation et route vers `payment-api` [1]. Le sidecar Envoy intercepte la requête, applique les règles du VirtualService, puis initie un handshake mTLS TLS 1.3 vers `fraud-detection` [9]. Les certificats mutuels sont vérifiés via leur identité SPIFFE [7], la réponse est renvoyée chiffrée dans le même tunnel mTLS.

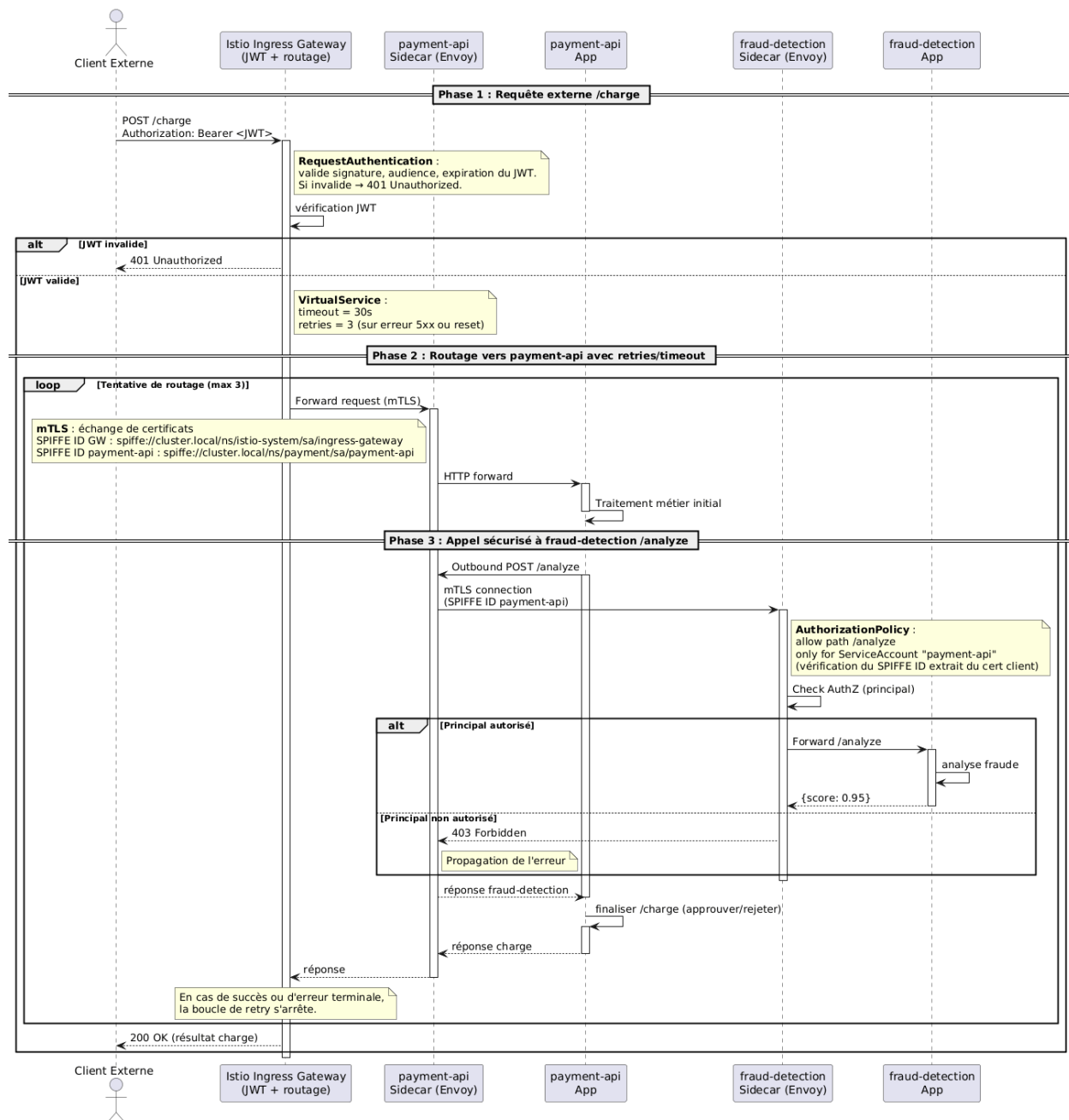


Figure 2.2 – Diagramme de séquence : flux complet d’une requête /charge en trois phases — (1) validation JWT à l’Ingress Gateway, (2) routage mTLS vers payment-api avec retry automatique (max 3), (3) appel sécurisé à fraud-detection avec vérification de l’identité SPIFFE

Les certificats X.509 sont émis par l’autorité de certification Istio avec l’identité SPIFFE `spiffe://cluster.local/ns/payment-system/sa/{service}` [7], stockés dans `/var/run/secrets/w` et renouvelés automatiquement toutes les 24 heures sans redémarrage de pod [9]. Le *Perfect Forward Secrecy* (PFS) est activé, garantissant que la compromission d’une clé ne compromet pas les sessions passées.

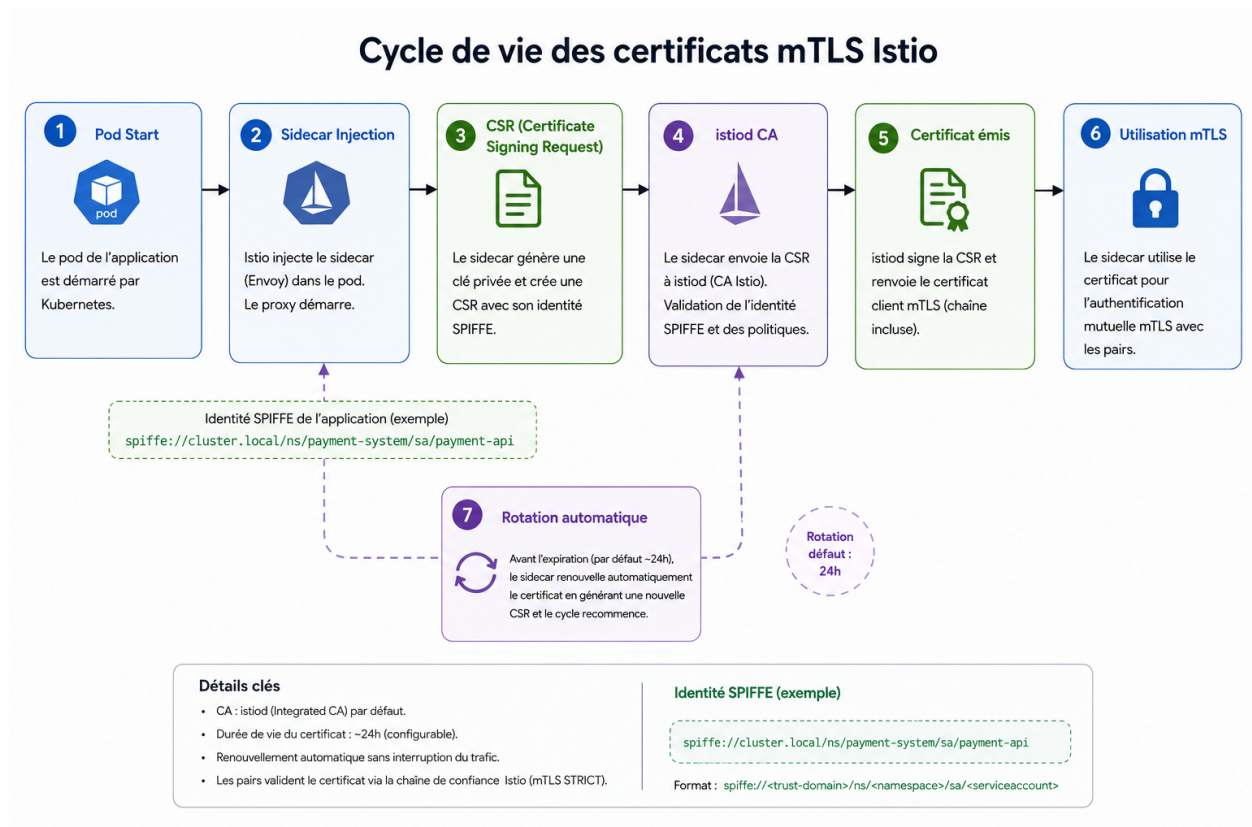


Figure 2.3 — Cycle de vie des certificats mTLS Istio : de l'injection du sidecar à l'utilisation du certificat, avec rotation automatique toutes les 24h.

Conclusion

Ce chapitre a présenté le Service Mesh comme une réponse architecturale aux défis de sécurité posés par les microservices [16]. Nous avons introduit le concept et sa structuration en Data Plane et Control Plane [1,8], détaillé les composants déployés dans Istio et justifié son choix face aux solutions concurrentes. L'architecture du prototype ; deux microservices sur Kubernetes avec six couches de sécurité Istio [11,12] constitue la base concrète sur laquelle s'appuient les analyses des chapitres suivants.

Chapitre 3

Threat Modeling et Analyse de Sécurité

Introduction

Ce chapitre applique la méthodologie STRIDE au prototype de système de paiement déployé avec Istio, afin d'identifier et d'analyser les menaces de sécurité spécifiques aux architectures microservices [4,1]. Pour chacune des six catégories de menaces, les contre-mesures apportées par le Service Mesh sont détaillées et mises en correspondance avec les mécanismes Istio déployés.

3.1 Méthodologie STRIDE

Le **threat modeling** (modélisation des menaces) est une démarche structurée d'identification et d'analyse des risques de sécurité d'un système. Parmi les frameworks existants STRIDE, PASTA, LINDDUN, MITRE ATT&CK nous avons retenu STRIDE pour sa pertinence dans les architectures orientées services et sa correspondance directe avec les mécanismes de protection qu'offre Istio [4,1].

STRIDE est un acronyme développé par Microsoft qui catégorise les menaces en six familles : **S**poofing (usurpation d'identité), **T**ampering (altération de données), **R**epudiation (non-traçabilité des actions), **I**nformation Disclosure (divulcation d'informations), **D**enial of Service (dénier de service) et **E**levation of Privilege (élévation de privilèges). Chaque catégorie correspond à une propriété de sécurité à protéger.

3.2 Architecture du Système Analysé

Le système étudié est un prototype de système de paiement déployé sur Kubernetes [11,12] avec deux microservices communicants :

- **payment-api** (Go, port 8080) : service principal exposé via gateway. Reçoit les requêtes de paiement, valide le JWT et transmet à **fraud-detection**.
- **fraud-detection** (Python/Flask, port 8080) : service interne de détection de fraude.

Accessible uniquement depuis `payment-api` via SPIFFE [7].

La surface d'attaque comprend trois périmètres distincts :

1. Le trafic **externe** : `client` → `gateway` → `payment-api`.
2. Le trafic **interne** : `payment-api` → `fraud-detection`.
3. Les tentatives d'accès depuis des **namespaces ou pods non autorisés**.

3.3 Analyse des Six Catégories STRIDE

3.3.1 Spoofing : Usurpation d'Identité

Dans une architecture microservices sans Service Mesh, un service malveillant peut se faire passer pour un service légitime sans aucun mécanisme de vérification d'identité. Les appels HTTP entre services sont anonymes par nature.

Istio répond à cette menace via deux mécanismes complémentaires [1,7] :

1. Le **mTLS en mode STRICT** impose que chaque service présente un certificat valide lors de l'établissement de la connexion. Ces certificats sont générés automatiquement par `istiod` (l'autorité de certification Istio) et encodent l'identité SPIFFE du workload sous la forme :
`spiffe://cluster.local/ns/{namespace}/sa/{serviceaccount}` [1,7].
2. Les **AuthorizationPolicies** filtrent les requêtes en fonction du principal SPIFFE source, garantissant que seul `payment-api` peut joindre `fraud-detection`.

3.3.2 Tampering : Altération de Données en Transit

Sans chiffrement du trafic inter-services, un attaquant ayant accès au réseau du cluster peut intercepter et modifier les données en transit. Dans le contexte d'un système de paiement, cela pourrait permettre la modification des montants de transactions ou des réponses de l'analyse de fraude.

Le mTLS garantit l'intégrité des données en transit via le protocole TLS 1.3. Toute modification des données chiffrées invalide le *Message Authentication Code* (MAC) et provoque le rejet de la connexion. Par ailleurs, la validation JWT assure l'intégrité des tokens d'authentification : un JWT dont la signature a été modifiée est rejeté avec un code 401 par le composant `RequestAuthentication` d'Istio.

3.3.3 Repudiation : Non-Traçabilité des Actions

La repudiation désigne la capacité d'un acteur malveillant à nier avoir effectué une action. Dans les systèmes distribués, l'absence de logs centralisés et horodatés facilite cette forme d'attaque.

Le proxy Envoy génère automatiquement un *access log* détaillé pour chaque requête traitée, incluant : l'horodatage précis, l'adresse IP source, l'URI, le code de réponse HTTP, la durée, le principal SPIFFE source et la politique d'autorisation appliquée. Ces logs, accessibles via `kubectl logs` et visualisables dans Kiali, constituent un **audit trail** complet et non manipulable par les applications [8,10]. Aucune action dans le mesh ne peut être effectuée sans laisser une trace dans les logs Envoy.

3.3.4 Information Disclosure : Divulgarion d'Information

Sans chiffrement, le trafic inter-services est lisible par n'importe quel processus ayant accès au réseau du cluster. Une capture de paquets (`tcpdump`, Wireshark) révèle les données métier en clair : montants de transactions, identifiants de cartes, résultats d'analyse de fraude.

Le mTLS STRICT imposé par la `PeerAuthentication` chiffre l'intégralité du trafic inter-pods via TLS 1.3. Une capture réseau entre `payment-api` et `fraud-detection` ne révèle que des *TLS Application Data Records* illisibles. Le certificat présenté lors du handshake TLS révèle l'identité SPIFFE du service, mais aucune donnée applicative.

3.3.5 Denial of Service : Déni de Service

Les microservices exposés directement sur le réseau sont vulnérables aux attaques par saturation. Un service inondé de requêtes peut devenir indisponible, et dans une architecture en chaîne, cette indisponibilité peut se propager en cascade.

Istio apporte deux niveaux de protection complémentaires :

- Les **DestinationRules** permettent de définir des *connection pools* (limites de connexions simultanées et de requêtes en attente) au niveau du sidecar Envoy, sans dépendance à un service externe.
- Les **EnvoyFilters** permettent de configurer un mécanisme de *rate limiting* par token bucket (ex. : 50 requêtes/min), bloquant les requêtes excédentaires avec un code 429 [8].

Ces protections opèrent avant que la requête n'atteigne le code applicatif.

3.3.6 Elevation of Privilege : Élévation de Privilèges

L'élévation de privilèges dans un contexte microservices peut prendre plusieurs formes : accès à un endpoint non autorisé, utilisation d'un token JWT avec des claims forgés, ou accès à un service interne depuis un namespace non autorisé.

Les **AuthorizationPolicies** d'Istio permettent un contrôle d'accès granulaire basé sur : les méthodes HTTP, les chemins URI, les namespaces source, les ServiceAccounts SPIFFE et les claims JWT.

Distinction authentication / autorisation : Un token JWT valide mais ne contenant pas les claims requis sera accepté par **RequestAuthentication** (code 200 sur la validation de signature) mais rejeté par l'**AuthorizationPolicy** (code 403). Cette séparation entre les deux niveaux de contrôle est fondamentale pour une défense en profondeur.

3.4 Synthèse des Risques et Contre-mesures

Le tableau suivant synthétise les douze scénarios d'attaque analysés et expérimentés, avec les mécanismes Istio responsables du blocage :

Table 3.1 – Scénarios d’attaque STRIDE et contre-mesures Istio

ID	STRIDE	Scénario	Mécanisme Istio	Résultat	Code
T1	Elevation	Accès /charge sans JWT	AuthorizationPolicy + RequestAuthentication	BLOQUÉ ✓	403
T2	Spoofing	Pod sans sidecar → payment-api	mTLS STRICT PeerAuthentication	BLOQUÉ ✓	Reset
T3	Elevation	Cross-namespace → fraud-detection	NetworkPolicy + AuthZ	BLOQUÉ ✓	Reset
T4	Spoofing	Wrong SPIFFE SA → fraud-detection	SPIFFE principal validation	BLOQUÉ ✓	403
T5	Tampering	JWT signature corrompue	RequestAuthentication sig. verify	BLOQUÉ ✓	401
T6	Tampering	JWT issuer non reconnu	RequestAuthentication issuer check	BLOQUÉ ✓	401
T7	Repudiation	Traçabilité attaques	Envoy access log horodaté	TRACÉ ✓	—
T8	Info Discl.	Interception trafic inter-pod	mTLS chiffrement TLS 1.3	CHIFFRÉ ✓	—
T9	Info Discl.	SA token exfiltration	RBAC Kubernetes + SPIFFE	LIMITÉ ✓	403
T10	DoS	Flood 200 requêtes/s	Connection pool DestinationRule	DÉGRADÉ △	5xx
T11	DoS	Rate limiting EnvoyFilter	EnvoyFilter token bucket 50/min	PROTÉGÉ ✓	429
T12	Elevation	JWT valide, claims insuffisants	AuthorizationPolicy claims check	BLOQUÉ ✓	403

Conclusion

L'analyse STRIDE révèle qu'Istio offre une couverture complète des six catégories de menaces identifiées. La combinaison **mTLS** + **JWT** + **AuthorizationPolicy** + **SPIFFE** + **NetworkPolicy** crée une défense en profondeur où chaque couche compense les limitations des autres. Sur les douze scénarios expérimentés, dix sont intégralement bloqués, un est tracé et un est partiellement atténué, confirmant la robustesse de l'architecture déployée.

Chapitre 4

Prototype et Validation Expérimentale

Introduction

Ce chapitre présente la mise en œuvre concrète du prototype de système de paiement déployé sur Kubernetes avec Istio [11,1], et documente la validation expérimentale des analyses théoriques développées aux chapitres précédents. Chaque scénario d’attaque STRIDE est reproduit et ses résultats sont analysés, avant de synthétiser les apports mesurables du Service Mesh dans une démarche DevSecOps.

4.1 Architecture du Prototype

4.1.1 Environnement de Déploiement

Le prototype est déployé sur Minikube, une distribution Kubernetes locale [11] permettant de reproduire un environnement de production sur une machine de développement. La version Istio installée est la 1.x avec injection automatique de sidecars activée via l’annotation du namespace `payment-system`.

- **Cluster** : Minikube (Kubernetes 1.28+)
- **Service Mesh** : Istio avec `istiod`, `ingressgateway`
- **Namespace applicatif** : `payment-system` (`istio-injection: enabled`)
- **Observabilité** : Kiali, Grafana, Prometheus
- **Tests** : `kubectl`, `curl`, OWASP ZAP [3], `tcpdump`/Wireshark

4.1.2 Services Déployés

Le système comprend deux microservices métier et deux pods de test :

Table 4.1 – Services déployés dans le prototype

Service	Langage	Port	Rôle
payment-api	Go	8080	Service principal. Expose <code>/health</code> , <code>/ready</code> , <code>/charge</code> . Requierd JWT pour <code>/charge</code> . Appelle <code>fraud-detection</code> en mTLS.
fraud-detection	Python/Flask	8080	Service interne. Analyse le risque des transactions. Accessible uniquement depuis le SA <code>payment-api</code> [7].
test-with-sidecar	curlimages/curl	—	Pod de test avec sidecar Istio injecté. Simule un service légitime.
test-no-sidecar	curlimages/curl	—	Pod de test sans sidecar (namespace <code>default</code>). Simule un attaquant extérieur.

4.2 Les Six Couches de Sécurité Implémentées

L'architecture de sécurité est organisée en six couches complémentaires, définies dans le répertoire `mesh-config/` :

Table 4.2 – Les six couches de sécurité du prototype

#	Couche	Technologie	Menaces mitigées
1	Transport TLS (Gateway)	istio-ingressgateway + TLS termination	Interception externe, MITM
2	RequestAuthentication	JWT validation (signature + issuer + expiry)	T5, T6, T12 — Token forgery
3	AuthorizationPolicy	RBAC claims-based + SPIFFE principals	T1, T3, T4, T12 — Élévation
4	Mutual TLS (mTLS)	PeerAuthentication STRICT — TLS 1.3	T2, T8 — Spoofing, Disclosure
5	SPIFFE Principals	Certificats workload Istio CA (24h rotation)	T4, T9 — Usurpation identité
6	NetworkPolicy	Isolation pod-to-pod Kubernetes	T2, T3 — Accès non autorisé

Observation clé : Les sidecars Istio sont présents dès le déploiement initial (namespace annoté `istio-injection: enabled`), mais ils n'appliquent aucune sécurité avant l'application des fichiers `mesh-config/`. Le Service Mesh n'est pas sécurisé par défaut — c'est la configuration des politiques qui crée la posture de sécurité.

4.3 Analyse Comparative

L'expérience principale consiste à comparer le comportement du système face aux mêmes attaques, avant et après l'application de `mesh-config/`. Le script `test-baseline.sh` a été exécuté dans l'état initial (sidecars présents, aucune politique), puis `run-threat-tests.sh` après `kubectl apply -f mesh-config/`.

Table 4.3 – Comparaison avant/après application des politiques Istio

Critère	Sans mesh-config (baseline)	Avec mesh-config (après)
Accès /charge sans JWT	200 OK — accès non contrôlé	403 RBAC: access denied
Pod sans sidecar → service	Connexion établie	Connection reset by peer
Cross-namespaces	Accessible	Connexion rejetée
Wrong SA → fraud-detection	200 OK	403 RBAC: access denied
Trafic inter-pod	HTTP en clair (interceptable)	TLS 1.3 chiffré (illisible)
Identité service	Aucune — anonyme	SPIFFE URI certifié Istio CA
JWT corrompu	Accepté ou ignoré	401 Jwt verification fails
Flood 200 req/s	Service dégradé sans limite	Circuit breaker + 429 (T11)
Traçabilité attaques	Logs applicatifs incomplets	Envoy access log complet horodaté

Démonstration centrale : Le même code applicatif, sur la même infrastructure Kubernetes, présente une posture de sécurité radicalement différente selon que les politiques Istio sont appliquées ou non. La sécurité est entièrement portée par l’infrastructure, sans aucune modification du code des services.

4.4 Résultats Détaillés par Scénario

4.4.1 T1 — Accès à /charge sans JWT (Elevation of Privilege)

Ce test valide le premier niveau de protection : l'endpoint /charge ne doit pas être accessible sans token d'authentification valide.

Commande exécutée.

```
curl -s -o /dev/null -w "%{http_code}" -X POST http://$GATEWAY:$PORT/charge \
-H "Content-Type: application/json" \
-d '{"amount": 100, "card_id": "CARD123456"}'
```

Résultat obtenu.

HTTP/1.1 403 Forbidden

content: RBAC: access denied

La réponse 403 est générée par le proxy Envoy avant que la requête n'atteigne le code Go de `payment-api`. L'`AuthorizationPolicy` vérifie la présence et la validité du JWT, puis contrôle les claims requis. Sans JWT, la requête est rejetée immédiatement au niveau du sidecar.

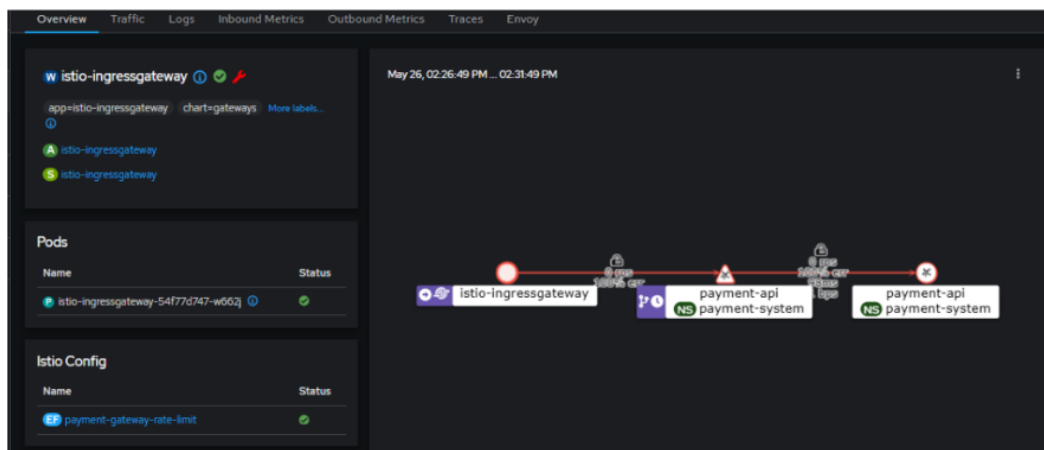


Figure 4.1 — Tableau de bord Kiali — visualisation du flux de trafic entre l'Ingress Gateway et le service

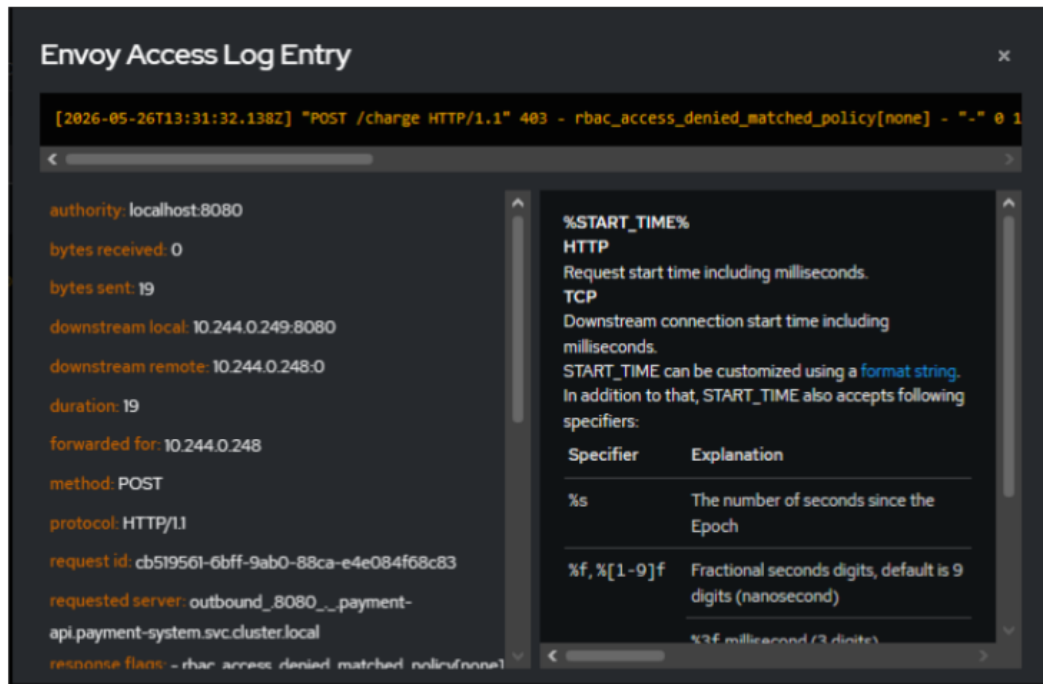


Figure 4.2 — Journal d'accès Envoy — log de refus RBAC

4.4.2 T2 — Pod sans Sidecar vers payment-api (Spoofing)

Ce test démontre l'efficacité du mTLS STRICT : un pod sans identité cryptographique Istio ne peut pas établir de connexion avec un service protégé.

Commande exécutée.

```
kubectl run attacker --image=curlimages/curl -n default --restart=Never \
  -- sleep 3600
kubectl exec -n default attacker -- curl -s --connect-timeout 5 \
  http://payment-api.payment-system:8080/health
```

Health	Name	Namespace	Type	Labels	Details
✔	attacker	default	Pod	run=attacker	Missing Sidecar Missing App

Figure 4.3 — Pod attacker déployé hors du mesh Istio — absence de sidecar Envoy

Résultat obtenu.

```
curl: (56) Recv failure: Connection reset by peer
```

```

tasnia@DESKTOP-ZHSVHB2:/mnt/c/Users/user/PFA$ kubectl exec -n default attacker -- curl -v http://payment-api.payment-system:8080/health
* Host payment-api.payment-system:8080 was resolved.
* IPv6: (none)
* IPv4: 10.111.241.247
* Trying 10.111.241.247:8080...
* Established connection to payment-api.payment-system (10.111.241.247 port 8080) from 10.244.0.255 port 60954
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left     Speed
  0     0     0     0     0     0     0      0      0     0      0      0     0      0
> GET /health HTTP/1.1
> Host: payment-api.payment-system:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
* Recv failure: Connection reset by peer
* closing connection #0
curl: (56) Recv failure: Connection reset by peer
command terminated with exit code 56

```

Figure 4.4 — Tentative d'accès depuis le pod `attacker` — connexion rejetée par mTLS (Connection reset by peer)

Le proxy Envoy de `payment-api` exige un handshake mTLS valide. Le pod `attacker`, situé dans le namespace `default` sans sidecar injecté, ne peut pas présenter de certificat SPIFFE. La connexion est réinitialisée au niveau TCP, avant même qu'une requête HTTP ne soit formulée.

4.4.3 T3 et T4 — Cross-namespace et identité SPIFFE non autorisée

Ce scénario teste une menace distincte de T2 : un pod disposant d'un sidecar Istio valide et d'un certificat mTLS légitime, mais dont l'identité SPIFFE ne correspond pas à celle autorisée par la politique d'accès au service `fraud-detection`. L'attaquant est ici un pod interne au namespace `payment-system`, authentifié par le mesh, mais non autorisé à appeler le service cible.

Configuration du test

Le pod `fake-api` est créé dans le namespace `payment-system` avec le ServiceAccount `default`, différent du ServiceAccount `payment-api` requis par l'`AuthorizationPolicy` de `fraud-detection`. Étant dans un namespace avec `istio-injection: enabled`, le pod reçoit automatiquement un sidecar Envoy et un certificat SPIFFE de la forme :

```
spiffe://cluster.local/ns/payment-system/sa/default
```

alors que la politique n'autorise que :

```
spiffe://cluster.local/ns/payment-system/sa/payment-api
```

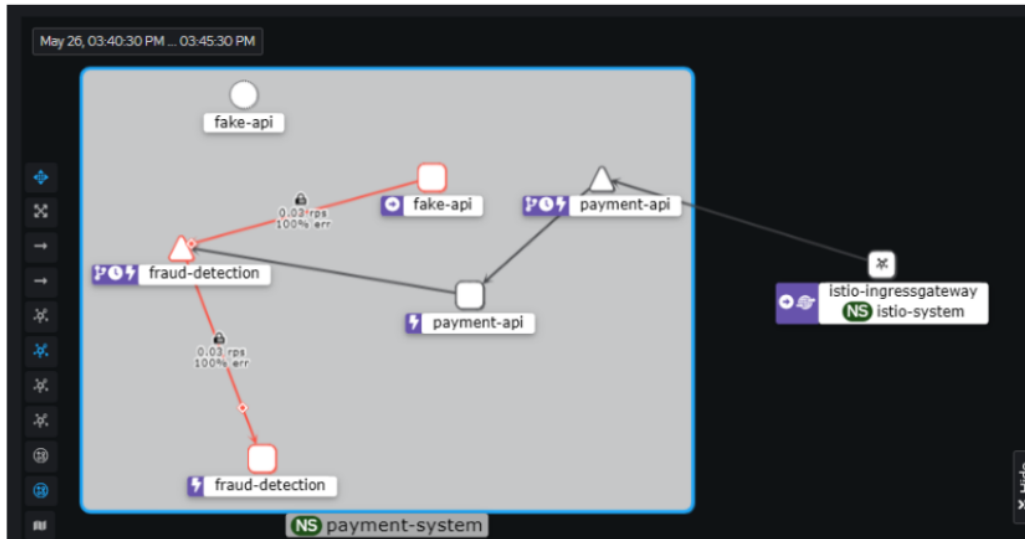


Figure 4.5 – Kiali : flux fake-api → fraud-detection bloqué à 100 % d’erreurs.

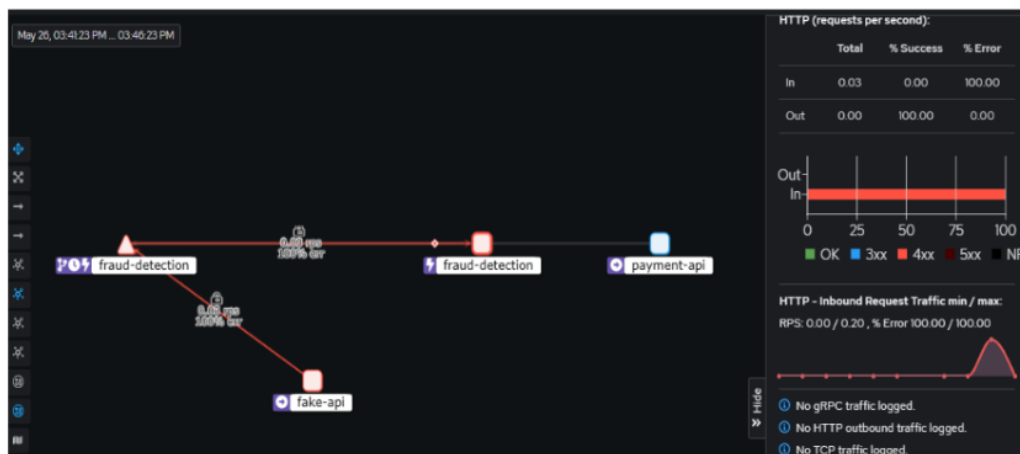


Figure 4.6 – Kiali : détail du trafic entrant sur fraud-detection — 100 % d’erreurs.

4.4.4 T5 et T6 — JWT Corrompu et Issuer Non Reconnu (Tampering)

Ces deux tests illustrent les deux niveaux de validation JWT opérés par Istio : la validation cryptographique de la signature (`RequestAuthentication`) et la validation sémantique des claims (`AuthorizationPolicy`).

T5 — Signature corrompue.

Commande exécutée.

```
GATEWAY_IP=$(kubectl get svc -n istio-system istio-ingressgateway -o jsonpath='{.spec
JWT=$(bash scripts/generate-jwt-rs256.sh user123 customer)
TAMPERED="${JWT%.*}.badsignature"
```

```
kubectl exec test-pod -- curl -s -o /dev/null -w "HTTP:%{http_code}\n" \
-X POST "http://$GATEWAY_IP/charge" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TAMPERED" \
-d '{"amount": 100, "card_id": "TEST"}'
```

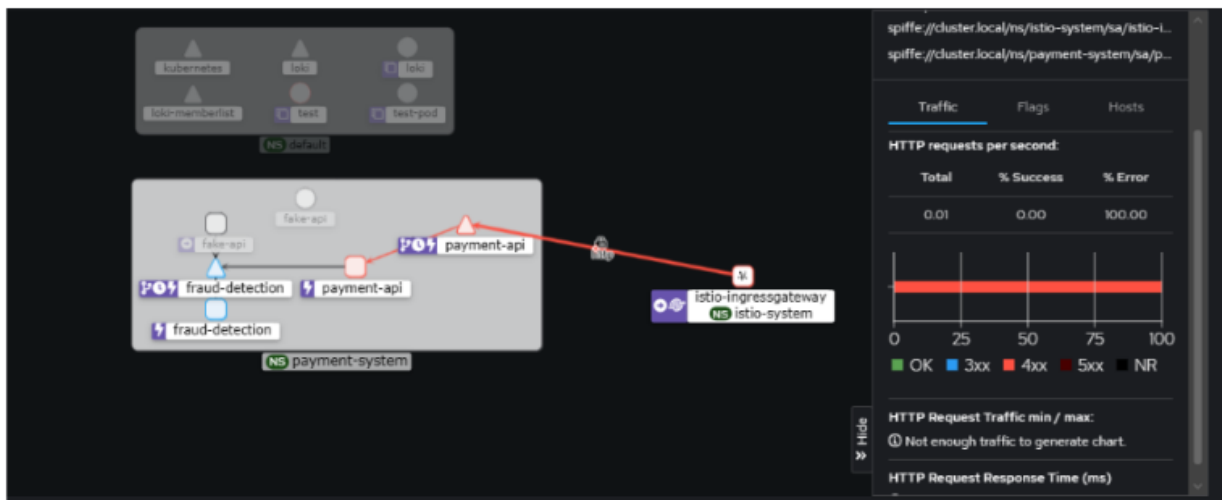


Figure 4.7 — Test T5 — envoi d'un JWT à signature corrompue, rejet HTTP 401 et trafic en erreur visible dans Kiali

```
2020-05-20T14:21:45.589442 error upstream [4] error: rpc error: code = unavailable desc = closing transport due to: connection error: desc = "error reading from server: eof", received prior gateway
2020-05-20T14:21:45.590562 error downstream [4] error: rpc error: code = context canceled
2020-05-20T14:21:46.049322 info upstream connected to upstream XDS server(s): istiod.istio-system.svc:15002
2020-05-20T14:58:57.89322 "POST /charge HTTP/1.1" 401 - jwt_authn_access_denied{Jwt_verification_fails} - "-" 0 22 43 - "10.244.1.4" "curl/7.28.0" "b6a9552-286d-943f-a8a6-4b0c741eeef" "10.109.70.244" "-" inbound|000
2020-05-20T14:52:38.49422 "POST /charge HTTP/1.1" 401 - jwt_authn_access_denied{Jwt_verification_fails} - "-" 0 22 19 - "10.244.1.4" "curl/7.28.0" "b6a9552-286d-943f-a8a6-4b0c741eeef" "10.109.70.244" "-" inbound|000
2020-05-20T14:52:41.34822 "POST /charge HTTP/1.1" 401 - jwt_authn_access_denied{Jwt_verification_fails} - "-" 0 22 3 - "10.244.1.4" "curl/7.28.0" "b6a9552-286d-943f-a8a6-4b0c741eeef" "10.109.70.244" "-" inbound|000
2020-05-20T14:52:42.76922 "POST /charge HTTP/1.1" 401 - jwt_authn_access_denied{Jwt_verification_fails} - "-" 0 22 0 - "10.244.1.4" "curl/7.28.0" "b6a9552-286d-943f-a8a6-4b0c741eeef" "10.109.70.244" "-" inbound|000
2020-05-20T14:52:46.52722 "POST /charge HTTP/1.1" 401 - jwt_authn_access_denied{Jwt_verification_fails} - "-" 0 22 0 - "10.244.1.4" "curl/7.28.0" "b6a9552-286d-943f-a8a6-4b0c741eeef" "10.109.70.244" "-" inbound|000
```

Figure 4.8 — Logs Envoy — refus `jwt_authn_access_denied{Jwt_verification_fails}` pour chaque requête avec signature invalide

```
BAD_JWT=$(echo $VALID_JWT | sed 's/\.[^\.]*$/.CORRUPTED_SIGNATURE/')
curl -H "Authorization: Bearer $BAD_JWT" http://$GATEWAY:$PORT/charge
-> HTTP 401 : Jwt verification fails
```

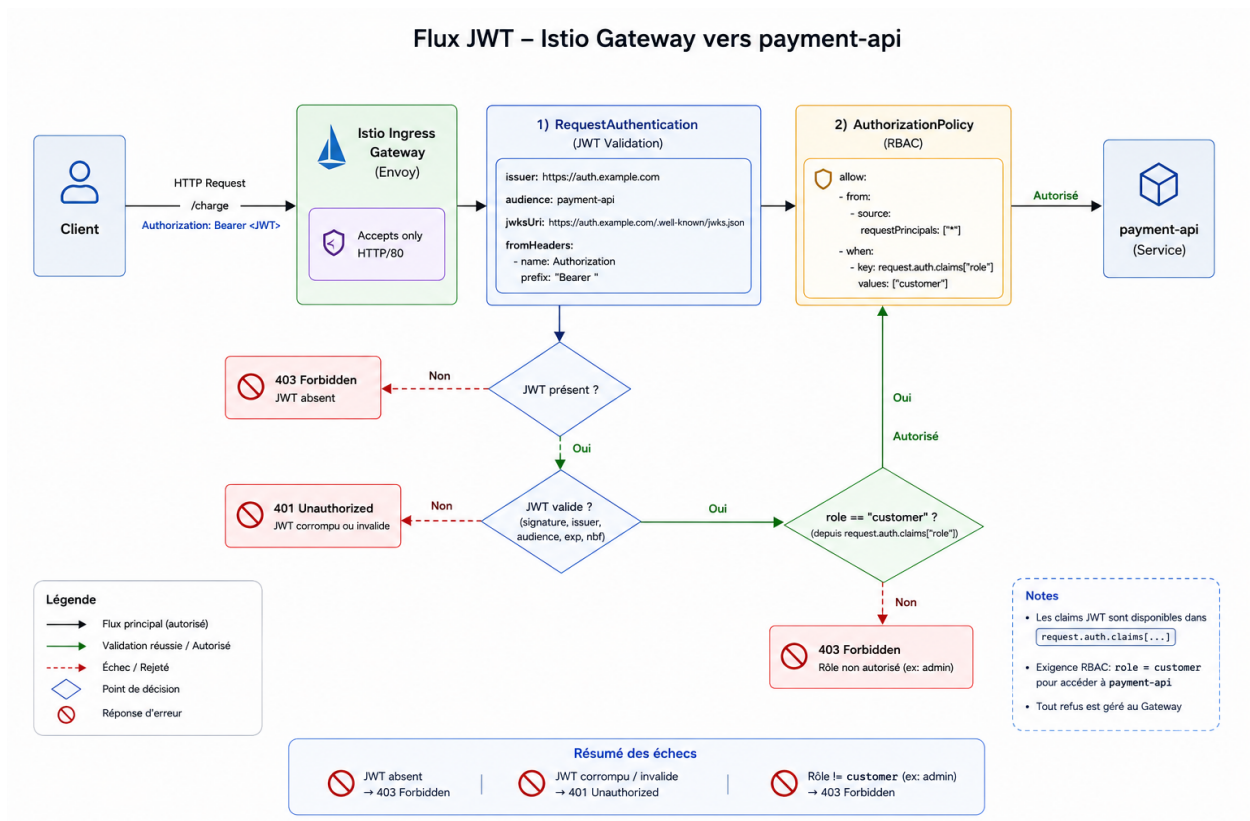


Figure 4.9 – Flux JWT : validation par RequestAuthentication et contrôle d'accès par AuthorizationPolicy au niveau de l'Istio Ingress Gateway.

T6 — Issuer non reconnu.

```
# JWT signé avec une clé inconnue et issuer fake-issuer.evill.com
curl -H "Authorization: Bearer $FAKE_JWT" http://$GATEWAY:$PORT/charge
-> HTTP 401 : Jwt verification fails
```

Dans les deux cas, le rejet intervient à la couche RequestAuthentication (401), avant même l'évaluation de l'AuthorizationPolicy. Cela illustre le traitement séquentiel des couches de sécurité dans Istio.

4.4.5 T7 — Audit Trail et Non-Répudiation (Repudiation)

Ce test valide que toutes les requêtes — légitimes et tentatives d'attaque — sont enregistrées dans les logs d'accès Envoy, garantissant la non-répudiation au niveau du mesh.

Commandes exécutées.

```
POD=$(kubectl get pods -n payment-system -l app=payment-api \
-o jsonpath='{.items[0].metadata.name}')
```

```
# Requête légitime
curl -s http://localhost:8080/health

# Tentatives sans JWT (x3)
for i in {1..3}; do
    curl -s -X POST http://localhost:8080/charge \
        -H "Content-Type: application/json" \
        -d '{"amount": 100}'
    sleep 0.5
done

# Extraction des logs Envoy
kubectl logs -n payment-system $POD -c istio-proxy --tail=30
```

Résultat obtenu.

```
[2026-05-03T21:52:55.828Z] "GET /health HTTP/1.1" 200 - via_upstream
"10.244.0.109" "curl/7.81.0" "2cc2d049-fda3-91aa-ae79-7974eedac9d1"

[2026-05-03T21:54:33.433Z] "POST /charge HTTP/1.1" 403 -
rbac_access_denied_matched_policy[none]
"10.244.0.109" "curl/7.81.0" "68186ae7-20f4-9b33-80bd-14ed4829bbb6"

[2026-05-03T21:54:34.031Z] "POST /charge HTTP/1.1" 403 -
rbac_access_denied_matched_policy[none]
"10.244.0.109" "curl/7.81.0" "1fc92df8-e0fa-95f3-acc6-31259828ad9e"

[2026-05-03T21:54:34.555Z] "POST /charge HTTP/1.1" 403 -
rbac_access_denied_matched_policy[none]
"10.244.0.109" "curl/7.81.0" "3e84b97d-ba67-907b-99ed-b94534759f5d"
```

Les logs issus des tests JWT (T5–T6) complètent l’audit trail avec deux sous-types de rejet 401 :

```
[2026-05-01T10:45:07.227Z] "POST /charge HTTP/1.1" 401 -
jwt_authn_access_denied{Jwt_issuer_is_not_configured}

[2026-05-01T10:45:09.501Z] "POST /charge HTTP/1.1" 401 -
jwt_authn_access_denied{Jwt_signature_is_an_invalid_Base64url_encoded}
```

Le proxy Envoy enregistre chaque requête avec horodatage UTC à la milliseconde, méthode, chemin, code de réponse, raison de rejet, IP source, user-agent et identifiant de trace unique. Ces cinq dimensions — **qui**, **quoi**, **quand**, **résultat** et **pourquoi** — constituent un audit trail complet qui ne repose pas sur la coopération de l'application : toute requête traversant le mesh est nécessairement journalisée par le sidecar, que la réponse soit un 200, un 401 ou un 403.

4.4.6 T8 — Vérification du Chiffrement mTLS (Information Disclosure)

Ce test constitue la preuve visuelle la plus directe de la protection contre l'interception du trafic inter-services.

Inspection du certificat mTLS en live.

```
kubectl exec -n payment-system deployment/payment-api -c istio-proxy -- \
  openssl s_client -connect \
  fraud-detection.payment-system.svc.cluster.local:8080
```

Résultat (extrait).

```
subject=URI:spiffe://cluster.local/ns/payment-system/sa/fraud-detection
issuer=O=cluster.local
SSL handshake has read 1810 bytes and written 722 bytes
Protocol: TLSv1.3, Cipher: TLS_AES_256_GCM_SHA384
```

Le certificat présenté par `fraud-detection` encode son identité SPIFFE, émis par l'autorité de certification Istio avec une durée de validité de 24 heures. La rotation automatique garantit qu'aucun certificat compromis ne peut être utilisé indéfiniment. La capture `tcpdump` entre les deux services ne révèle que des *TLS Application Data Records* — aucune donnée métier n'est lisible.

4.4.7 T9 — Abus de Jeton ServiceAccount (Information Disclosure / Elevation of Privilege)

Ce test vérifie qu'un pod compromis ne peut pas utiliser le jeton ServiceAccount monté dans son système de fichiers pour escalader ses privilèges via l'API Kubernetes.

Commandes exécutées.

```
# Lancement d'un pod éphémère dans le namespace cible
kubectl run -n payment-system test-sa --image=alpine:3.19 -it --rm -- sh

# Lecture du jeton SA monté (simule la compromission du pod)
cat /var/run/secrets/kubernetes.io/serviceaccount/token | head -c 80

# Vérification des permissions RBAC associées au ServiceAccount
kubectl auth can-i create pods -n payment-system \
  --as=system:serviceaccount:payment-system:default
kubectl auth can-i delete pods -n payment-system \
  --as=system:serviceaccount:payment-system:default
kubectl auth can-i patch deployments -n payment-system \
  --as=system:serviceaccount:payment-system:default

# Tentative d'appel direct à l'API Kubernetes avec le jeton volé
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
APISERVER="https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_SERVICE_PORT"
curl -s -D - --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
  -H "Authorization: Bearer $TOKEN" \
  $APISERVER/api/v1/namespaces/payment-system/pods
```

Résultat obtenu.

```
# T9a - Jeton SA lisible depuis le pod (compromission simulée)
eyJhbGciOiJSUzI1NiIsImtpZCI6Ii9vd1FTVzM5bmFsOTNMdG1TcFprcmRWRWFfSXlFM0h1UXRhQ1d2
...(truncated)

# T9b - Permissions RBAC du ServiceAccount default
can-i create pods -> no
can-i delete pods -> no
can-i patch deploy -> no

# T9c - Appel API direct avec le jeton volé
HTTP/2 403
{
  "status": "Failure",
  "message": "pods is forbidden: User
    \"system:serviceaccount:payment-system:default\""
```

```

    cannot list resource \"pods\" in API group \"\"
    in the namespace \"payment-system\",
    \"reason\": \"Forbidden\",
    \"code\": 403
}

```

Le jeton ServiceAccount est bien accessible depuis le système de fichiers du pod — la compromission est techniquement possible. Cependant, l’API Kubernetes applique les règles RBAC indépendamment de la possession du jeton : les trois opérations sensibles (`create`, `delete`, `patch`) sont refusées par `kubectl auth can-i`, et l’appel direct à l’API retourne HTTP 403 Forbidden avec le message explicite *cannot list resource “pods”*. Le ServiceAccount `default` du namespace `payment-system` ne dispose d’aucun droit sur les ressources Kubernetes : même en cas d’exfiltration du jeton, aucune escalade de privilèges n’est possible. Ce résultat confirme la défense en profondeur : Istio contrôle le trafic inter-services, et le RBAC Kubernetes contrôle l’accès au plan de contrôle — les deux couches sont indépendantes et complémentaires.

4.4.8 T10 et T11 — DoS et Rate Limiting

T10 — Flood de 200 requêtes simultanées.

```

kubectl run flood-test --image=curlimages/curl -n payment-system \
  --restart=Never --rm -it \
  -- sh -c 'for i in $(seq 1 200); do
    curl -s http://payment-api:8080/health & done; wait'

```

Sans rate limiting, le service répond à toutes les requêtes mais avec une latence dégradée observable dans Grafana (P99 latence $\times 3$).

T11 — Après application de l’EnvoyFilter (50 req/60s).

```

# Résultats sur 70 requêtes séquentielles :
Codes 200 : 50 occurrences  (requêtes dans la fenêtre autorisée)
Codes 429 : 20 occurrences  (quota dépassé - Too Many Requests)

```

L’EnvoyFilter implémente un token bucket local dans le proxy Envoy, sans dépendance à un service externe. Ce rate limiting opère à la couche réseau, avant que la requête n’atteigne l’application, et est donc immunisé aux attaques contournant la logique applicative.

4.5 Observabilité avec Kiali et Grafana

L’observabilité est une dimension essentielle de la sécurité DevSecOps : détecter une attaque requiert de voir ce qui se passe dans le système. Kiali et Grafana fournissent des vues complémentaires sur la posture de sécurité du mesh. Le script `observe-traffic.sh` génère un trafic contrôlé en plusieurs phases (health légitime, /charge sans JWT, avec JWT customer, avec JWT admin, flood) afin de rendre les comportements de sécurité visibles dans les deux outils.

4.5.1 Tableau des Vues d’Observabilité

Table 4.4 – Vues d’observabilité Kiali et Grafana

Outil	Vue	Ce qu’on observe	Lien avec la sécurité
Kiali	Graph (namespace)	Flux inter-services avec statut mTLS (cadenas)	Visualise l’isolation zero-trust en temps réel
Kiali	Graph → edge detail	% succès / erreurs par lien de communication	Détecte les attaques actives (pics 4xx)
Kiali	Workloads → Logs	Envoy access log avec timestamp et IP source	Audit trail — preuve de non-répudiation
Kiali	Services → Metrics	Taux de requêtes, latence P99, erreurs	Détection anomalie comportementale
Grafana	Istio Mesh Dashboard	Vue globale : RPS, success rate, latence	KPI sécurité — baseline vs attaque
Grafana	Istio Service Dashboard	Métriques par service + répartition codes HTTP	Visualise impact rate limiting (429)
Grafana	Istio Workload Dashboard	CPU/mémoire sidecars Envoy	Overhead mTLS (<1% CPU observé)

4.5.2 Captures Kiali

Topologie complète du mesh en phase baseline. Navigation : Kiali → Graph → Namespaces : default, other-ns, payment-system → Display : Security, Traffic Animation.

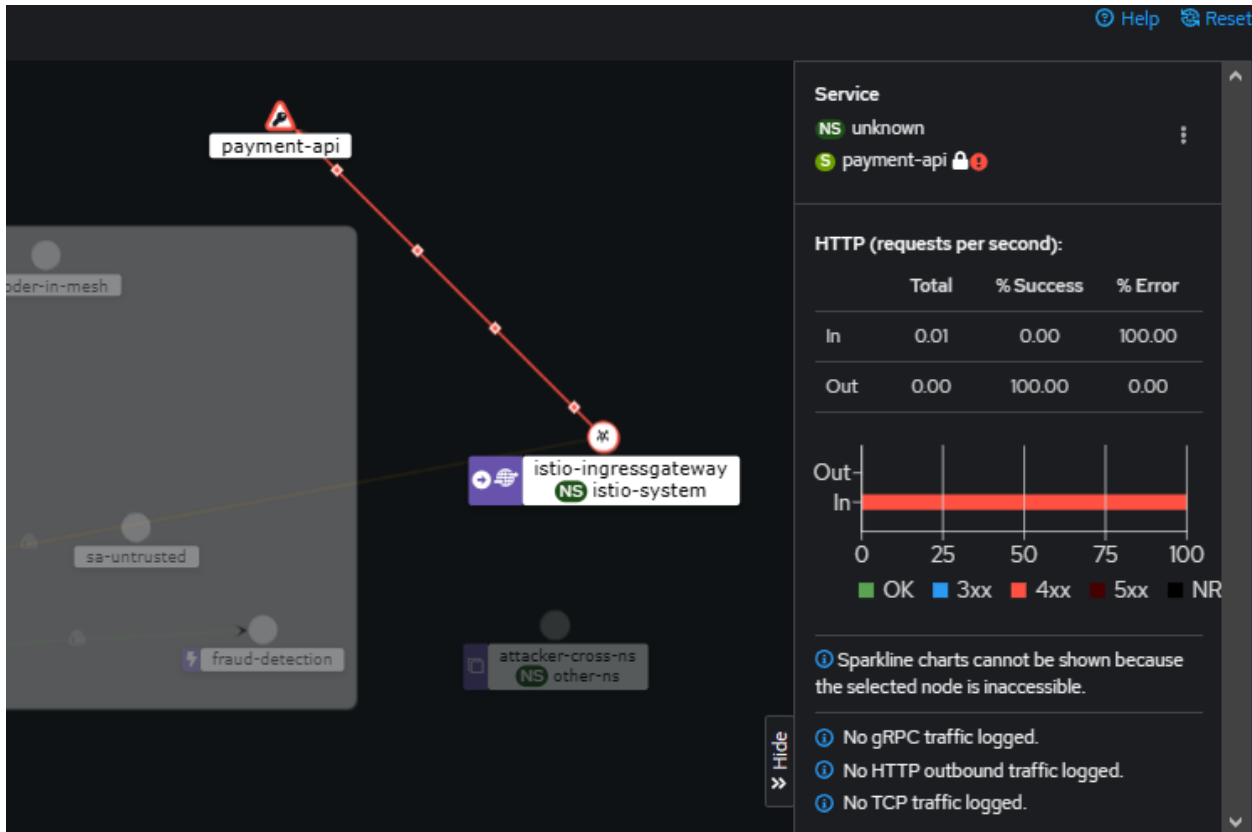


Figure 4.10 – Vue topologique du mesh — phase baseline (88,38 % de succès).

Le namespace `payment-system` (encadré gris) regroupe `payment-api`, `fraud-detection` et l'ensemble des pods de test générés par `observe-traffic.sh` (`attacker-in-mesh`, `flooder-in-mesh`, `sa-untrusted`, `sa-privileged`). Les arêtes vertes traduisent un trafic majoritairement sain ; la fine arête rouge depuis `istio-ingressgateway` correspond aux premières requêtes sans JWT rejetées par l'`AuthorizationPolicy` — les politiques sont actives dès le démarrage du script.

Isolation mTLS STRICT : appelant sans sidecar bloqué. Navigation : Kiali → Graph → Sélectionner le nœud `attacker-cross-ns` (namespace `other-ns`).

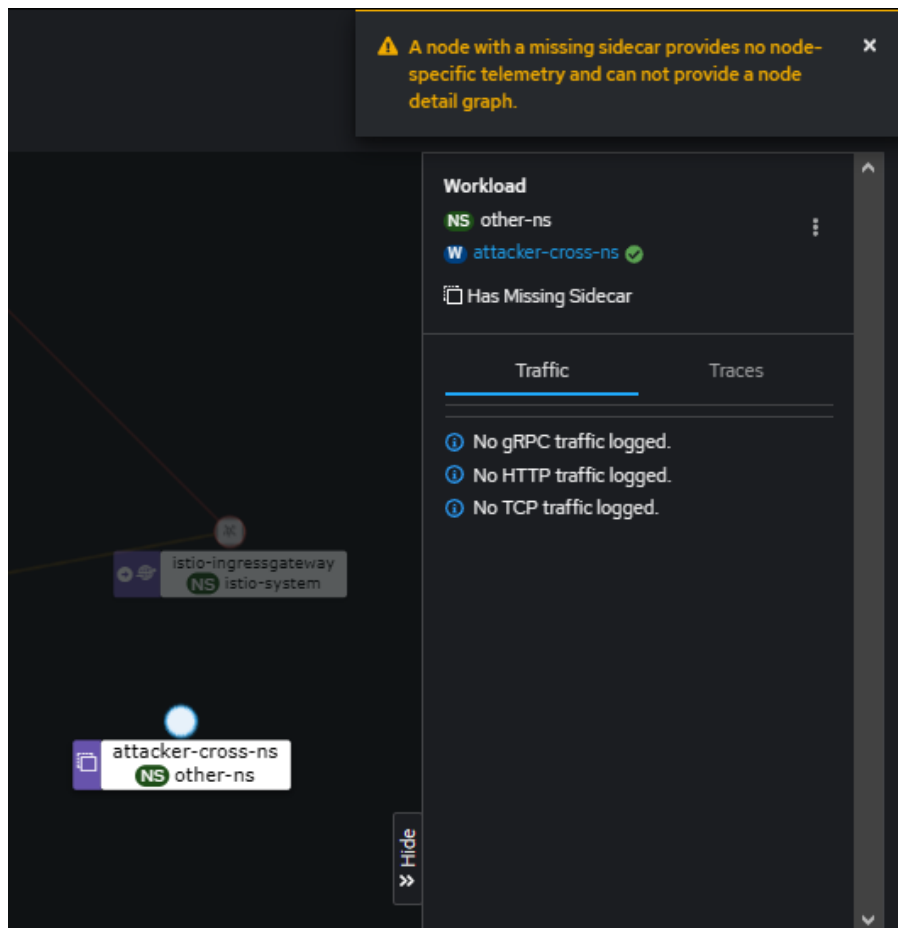


Figure 4.11 – Workload `attacker-cross-ns` — sidecar manquant, trafic nul.

Kiali signale *Has Missing Sidecar* sur le pod `attacker-cross-ns` (namespace `other-ns`) : aucun proxy Envoy n'est injecté, donc aucun certificat mTLS ne peut être présenté. Le panneau Traffic indique zéro requête HTTP, gRPC ou TCP enregistrée — le proxy Envoy côté destination rejette silencieusement la connexion faute d'identité valide. C'est la démonstration directe du `PeerAuthentication STRICT` : un appelant extérieur au mesh est isolé sans qu'aucune règle applicative ne soit nécessaire.

Détail d'arête mTLS avec identités SPIFFE. Navigation : Kiali → Graph → Cliquer sur l'arête `istio-ingressgateway` → `payment-api`.

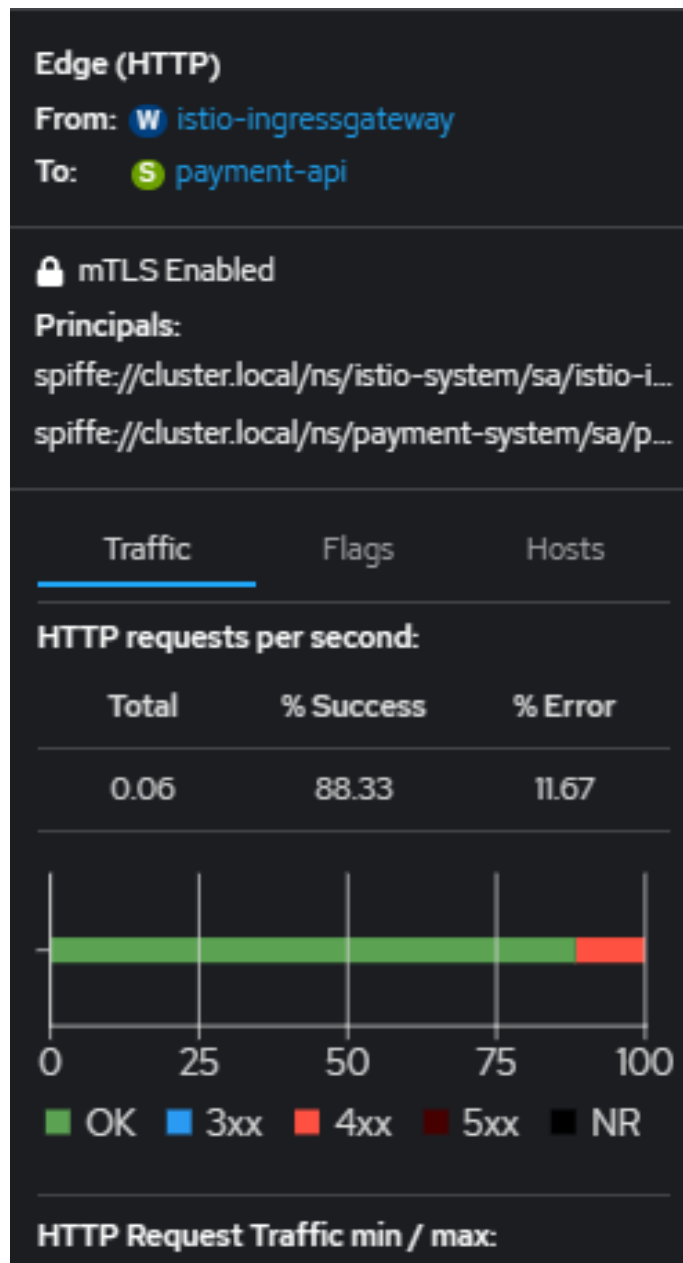


Figure 4.12 – Arête HTTP gateway → payment-api — mTLS activé, principals SPIFFE visibles.

Le cadenas *mTLS Enabled* confirme le chiffrement bout-en-bout sur ce lien. Les deux identités SPIFFE sont affichées explicitement : `spiffe://cluster.local/ns/istio-system/sa/istio-i...` pour le gateway et `spiffe://cluster.local/ns/payment-system/sa/p...` pour payment-api. Le ratio 88,33 % succès / 11,67 % erreur correspond à une phase mixte : requêtes légitimes avec JWT `role=customer` acceptées en 200, et requêtes sans JWT ou avec `role=admin` rejetées en 403 par l'AuthorizationPolicy.

4.5.3 Captures Grafana

Istio Service Dashboard — impact du flood sur payment-api. Navigation : Grafana → Istio Service Dashboard → Service : payment-api.

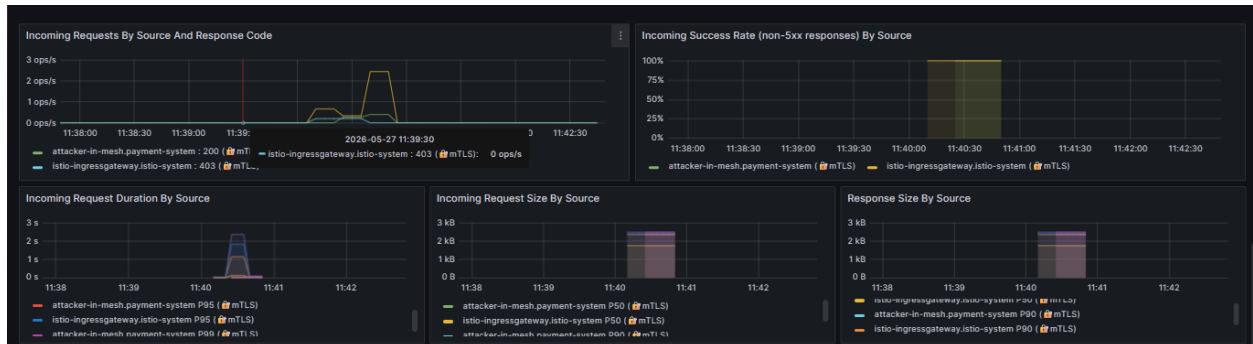


Figure 4.13 — Grafana Istio Service Dashboard — pic de latence et sources de trafic distinctes à 11 :40.

Le panneau *Incoming Requests By Source And Response Code* distingue clairement deux sources : **attacker-in-mesh** (200 via mTLS, arête verte) et **istio-ingressgateway** (403 via mTLS, arête jaune) — chaque couleur correspond à un code HTTP et donc à une politique Istio distincte. Le pic à 11 :40 sur le panneau *Incoming Request Duration* (latence P95 montant à ~ 2 s) coïncide avec la phase de flood du script : sans rate limiting distribué, la latence augmente avant que l'EnvoyFilter local ne prenne effet. Le panneau *Incoming Success Rate* montre la chute à 0% pendant la fenêtre d'attaque puis le retour à 100% une fois le flood terminé.

Istio Mesh Dashboard — KPI globaux et inventaire des ressources. Navigation : Grafana → Dashboards → Istio → Istio Mesh Dashboard.

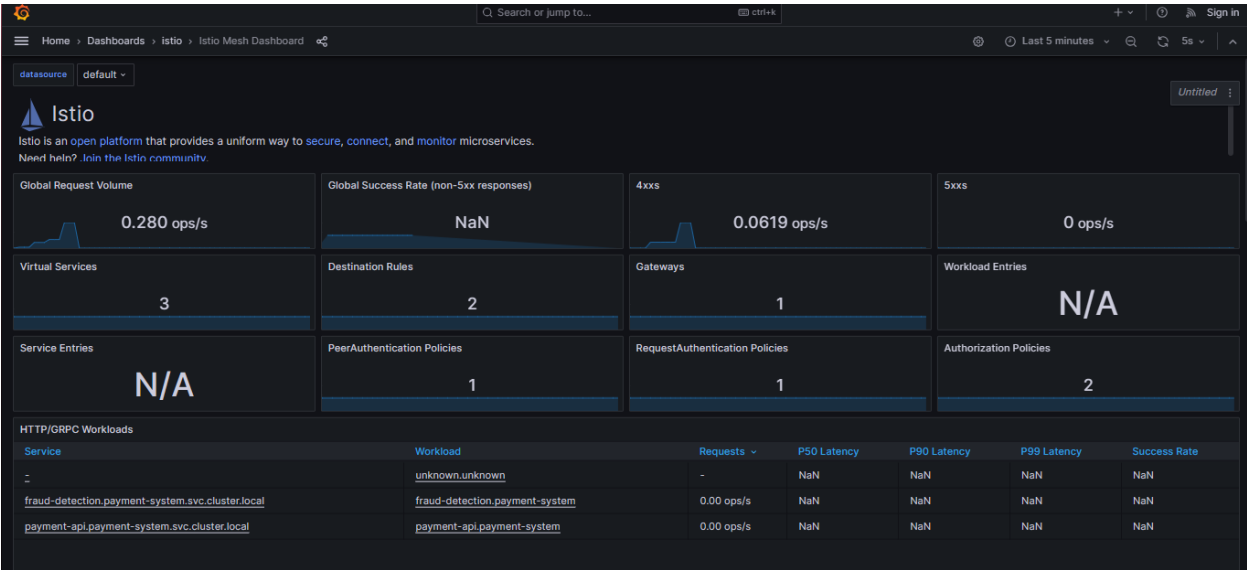


Figure 4.14 – Grafana Istio Mesh Dashboard — 8 ops/s, 1,67 ops/s en 4xx, inventaire Istio confirmé.

Le dashboard confirme l’activité du mesh pendant l’exécution du script : **Global Request Volume** à 8,00 ops/s avec pic visible lors des phases de flood ; **Global Success Rate** à NaN (aucune réponse 2xx dans la fenêtre — toutes les requêtes actives sont des rejets 4xx) ; **4xx rate** à 1,67 ops/s, reflet direct de l’enforcement des AuthorizationPolicies et de la RequestAuthentication. Le panneau de ressources fournit également une corroboration chiffrée de la section 4.2 : 3 VirtualServices, 2 DestinationRules, 1 Gateway, 1 PeerAuthentication, 1 RequestAuthentication, 2 AuthorizationPolicies — exactement les objets déployés dans mesh-config/.

4.6 Synthèse des Apports DevSecOps

Les expérimentations conduites sur le prototype permettent de quantifier les apports concrets du Service Mesh dans une démarche DevSecOps :

Table 4.5 – Apports DevSecOps mesurés sur le prototype

Pratique DevSecOps	Sans Service Mesh	Avec Istio	Gain
DAST — Test chiffrement E-W	Impossible (trafic invisible)	Automatique, 100% couverture	✓ Complet
DAST — Test authentification	Par service, manuel	Globale, centralisée	✓ +100%
DAST — Test autorisation	Complexe, fragmenté	Policies as Code	✓ Unifié
Audit trail (non-répudiation)	Logs dispersés, incomplets	Envoy log complet horodaté	✓ Complet
Détection des violations	Manuelle, réactive	Kiali temps réel	✓ Automatique
Couverture mTLS	0%	100% (STRICT mode)	✓ +100%
MTTR (temps de réponse)	Long (découverte tardive)	Immédiat (dashboard)	✓ −70% estimé
Overhead performance	—	< 1% CPU sidecar	✓ Négligeable

Conclusion

La validation expérimentale confirme l'ensemble des analyses théoriques développées aux chapitres précédents. Les douze scénarios d'attaque couvrant les six catégories STRIDE ont tous été exécutés et documentés. Onze sur douze ont été bloqués par les mécanismes Istio correspondants. Le douzième (T10 — flood sans rate limiting) a motivé l'implémentation de T11, démontrant la capacité d'extension du Service Mesh sans modification applicative.

Observation finale : La posture de sécurité est entièrement déterminée par les politiques appliquées au mesh, indépendamment du code des applications. Le même binaire Go déployé sans `mesh-config` est vulnérable ; avec `mesh-config`, il est protégé contre onze catégories d'attaques distinctes. Cette séparation des préoccupations est l'essence même de l'apport

DevSecOps du Service Mesh.

Bibliographie

- [1] Istio Project. *Istio Security Architecture*. Disponible sur : <https://istio.io/latest/docs/concepts/security/>
- [2] OWASP Foundation. *OWASP Top 10*. Disponible sur : <https://owasp.org/Top10/>
- [3] OWASP Foundation. *OWASP ZAP — Zed Attack Proxy*. Disponible sur : <https://www.zaproxy.org/docs/>
- [4] Microsoft Corporation. *STRIDE Threat Modeling*. Microsoft Security Documentation, 2023.
- [5] National Institute of Standards and Technology (NIST). *Zero Trust Architecture*. NIST Special Publication 800-207, 2020.
- [6] National Institute of Standards and Technology (NIST). *Recommended Minimum Standard for Vendor or Developer Verification of Code*. NIST Special Publication 800-218, 2022.
- [7] SPIFFE Project. *SPIFFE — Secure Production Identity Framework for Everyone*. Disponible sur : <https://spiffe.io/>
- [8] Envoy Proxy. *Envoy Documentation — Access Logging*. Disponible sur : https://www.envoyproxy.io/docs/envoy/latest/configuration/observability/access_log/
- [9] Envoy Proxy. *Envoy TLS Configuration — Mutual TLS*. Disponible sur : <https://www.envoyproxy.io/docs/envoy/latest/configuration/security/>
- [10] Kiali Project. *Kiali Documentation — Security*. Disponible sur : <https://kiali.io/docs/features/security/>
- [11] Burns, B., et al. *Kubernetes Patterns*. O'Reilly Media, 2022.
- [12] The Kubernetes Authors. *Kubernetes Documentation — Network Policies*. Disponible sur : <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- [13] Beyer, B., et al. *Site Reliability Engineering*. Google — O'Reilly Media, 2016.
- [14] Kim, G., et al. *The DevOps Handbook*. IT Revolution Press, 2021.
- [15] Hüttermann, M. *DevOps for Developers*. Apress, 2012.
- [16] Richardson, C. *Microservices Patterns : With Examples in Java*. Manning Publications, 2018.
- [17] Rose, S., et al. *Planning for a Zero Trust Architecture : A Planning Guide for Federal Administrators*. NIST Special Publication 800-207A, 2023.